SystemC based NoC Final Report

Arjun Behl, 501094790

Department of Computer and Electrical Engineering

Toronto Metropolitan University

Abstract

This report details the modifications and enhancements applied to a baseline SystemC Network-on-Chip (NoC) simulator, originally designed for a 1x2 topology. The primary objectives of this work were to extend the simulator's capabilities to model a larger, more complex 4x4 mesh topology, introduce interactive configuration for traffic patterns, and refine the core routing logic within the arbiter module to accurately implement wormhole flow control using Dimension-Order Routing (XY). Key modifications include source enable/disable controls and dynamic destination assignment within the source module. Additionally, a complete overhaul of the `sc_main` function was performed to instantiate and connect a 16-node mesh network featuring user-defined communication pairs. A significant rewrite of the arbiter's core logic enables stateful packet routing based on header flit information and tail flit teardown, consistent with wormhole switching principles. The subsequent analysis evaluates the design choices made, identifies potential issues and discusses the overall impact of these changes on the simulator's functionality, flexibility, and adherence to fundamental NoC principles.

I. INTRODUCTION

As semiconductor technology continues to scale, the integration of hundreds or even thousands of processing cores, memory blocks, and specialized hardware accelerators onto a single System-on-Chip (SoC) has become a reality . For these complex systems, traditional bus-based communication architectures struggle to provide the necessary scalability, bandwidth, and energy efficiency. Consequently, Networks-on-Chip (NoCs) have emerged as the predominant communication paradigm for modern multicore SoCs. NoCs effectively adapt principles from large-scale computer networks by employing routers and links to establish a dedicated on-chip communication fabric. This architectural approach offers substantial advantages concerning performance scalability, design modularity, and the effective management of communication bottlenecks inherent in highly integrated systems.

The design of efficient NoCs necessitates careful consideration of numerous critical factors, including network topology (e.g., Mesh, Torus), routing algorithm (e.g., XY, adaptive), flow control mechanism (e.g., Wormhole, Virtual Cut-Through), and buffer sizing . Evaluating the intricate trade-offs between performance, power consumption, and silicon area associated with these design choices is paramount before committing to hardware

implementation.

This project commenced with a baseline SystemC NoC simulator modeling a rudimentary 1x2 topology. The work documented herein significantly enhances this foundation. The core objectives were: scaling the topology to a 4x4 mesh and a 4x4 torus; implementing interactive traffic configuration; refining the arbiter logic for accurate wormhole flow control; implementing standard XY dimension-order routing; and improving overall code structure and debuggability. This report provides a detailed analysis of the specific code changes implemented in the `source.cpp`, `arbiter.cpp`, and `main_noc.cpp` files to achieve these objectives.

II. BACKGROUND AND THEORETICAL CONCEPTS

A thorough understanding of the modifications implemented in the simulator requires familiarity with several fundamental concepts related to Networks-on-Chip.

A. NoC Topologies

*1) 2D Mesh:*The 4x4 two-dimensional mesh topology, implemented in the modified simulator, is a widely studied and frequently employed structure in NoC design. Its structure is characterized by routers arranged in a regular grid formation. Within this grid, each router typically establishes connections with its immediate neighbors (North, South, East, West) and connects to a local Processing Element (PE) through a dedicated local port. This topology offers the advantages of a simple, regular structure facilitating easier layout and providing good scalability for moderate-sized systems. However, it also presents disadvantages, primarily potentially longer path lengths and higher latency compared to topologies with more direct connections or wrap-around links (like Torus).

2) 2D Torus: Torus topology extends the mesh by adding wrap-around links connecting routers on opposite edges of the grid (e.g., the rightmost router in a row connects to the leftmost router in the same row, and the bottom router in a column connects to the top router in that column). This creates a network without distinct edges or corners from a connectivity perspective. The primary advantage of a torus is potentially shorter average and maximum path lengths compared to a mesh of the same dimensions.

B. Routing Algorithms: Dimension-Order Routing (XY)

Routing algorithms dictate the path a packet traverses from source to destination. Deterministic algorithms, like the XY routing implemented here, consistently select the same path for a given source-destination pair, contrasting with adaptive algorithms that adjust paths based on network conditions. Dimension-Order Routing (DOR) is common in k-ary n-cube topologies (meshes, tori), typically mandating routing fully along one dimension before turning to the next. XY routing, a specific DOR for 2D meshes, routes horizontally (X-dimension) first, then vertically (Y-dimension).

While XY Dimension-Order Routing is naturally deadlock-free on a mesh, its application on a torus requires modification and careful consideration. To

leverage the wrap-around links effectively, the routing logic must calculate the shortest path in each dimension, considering both the forward path and the wrap-around path.

C. Flow Control: Wormhole Switching

Flow control mechanisms manage network resource allocation (buffers, links). Packets are often segmented into smaller, fixed-size flits (header, body, tail). Wormhole switching, implemented here, requires only small flit buffers at each router. The header flit reserves output channels hop-by-hop, and subsequent flits follow this path pipeline-style. Resources are released only after the tail flit passes. This yields low latency and reduced buffer needs.

III. IMPLEMENTATION DETAILS

This section provides a detailed account of the specific modifications implemented within the primary SystemC source code files.

A. `source.cpp` Modifications

The `source` module underwent several significant behavioral modifications. External enable control was introduced via an `sc_in<bool> enable_source` port and a check within the main loop, allowing `sc_main` to selectively activate sources. External destination control was implemented using an `sc_in<sc_uint<4>> target_dest_id` port; the destination ID is now read from this signal for the header flit (`pkt_snt == 0`), while subsequent flits receive a destination of -1 which translates to a value 15 as the value of these subsequent flits won't be considered by the arbiter , aligning with wormhole principles. A packet injection limit was enforced, stopping the source after transmitting 5 flits using a check `if (pkt_snt >= 5)`. Flit structure logic (5th flit as tail) and timing (`pkt_clk` toggle, initial `wait(1)`) were retained. Finally, enhanced console output including the flit count was added for better debugging.

B. `arbiter.cpp` Modifications

1) 4x4 Mesh: The `arbiter` module was significantly modified to implement stateful XY wormhole routing. The XY routing logic was implemented, calculating the output port based on current router ID and destination ID (extracted from the `req` signal) only for header flits (identified by `!v_connected_input[X]`). The arbitration and grant logic was updated to check output port reservation status for header flits and to update the wormhole state (`v_connected_input`, `v_reserved_output`) upon granting a request. Connection teardown logic was added to reset the state variables when a tail flit (identified by `reqX.read()[5]`) is processed. The implementation critically relies on a specific 6-bit interpretation of the `req` signal from the FIFO: `[tail(5), empty(4), destY(3,2), destX(1,0)]`. Numerous `printf` statements were added for debugging, and the logic was replicated for all 5 input ports.

2) 4x4 Torus:The primary change within the `arbiter` module was the replacement of the mesh XY routing logic with shortest-path XY routing logic suitable for a torus. This logic includes

1. If the coordinates in the current dimension differ, the forward distance (`dist_fwd`) and backward/wrap-around distance (`dist_bwd`) are calculated using modulo 4 arithmetic (e.g.,

`dist_fwd_x = (int(dest_x) - int(current_x) + 4) % 4`).

2. The shorter path direction is chosen. If `dist_fwd <= dist_bwd`, the positive direction (East for X, South for Y) is selected; otherwise, the negative direction (West for X, North for Y) is chosen. This includes a tie-breaking rule favoring East/South when the distance is 2.

3. The corresponding output port request (`v_req[X]`) is set (1=Local, 2=North, 3=East, 4=South, 5=West).

C. `main_noc.cpp` Modifications

1) 4x4 mesh:The `sc_main` function was completely transformed. Interactive configuration was added using standard C++ libraries. The simulation topology was scaled to a 4x4 mesh, requiring instantiation of 16 sources, sinks, and routers, and `dummy` signals for terminating unconnected edge/corner ports. Comprehensive port binding logic connects sources/sinks to local router ports, interconnects routers in the mesh pattern, and binds edge ports to dummies. Pre-simulation configuration assigns unique source/sink IDs (0-15), initializes enable signals, and configures enabled sources with their target destinations based on user input. Standard simulation control and VCD tracing are included. Basic results reporting summarizes configured exchanges and total packets sent/received.

2) 4x4 Torus:The `sc_main` function was updated to establish the torus connectivity. While retaining the instantiation of 16 sources, sinks, and routers, and the internal mesh connections, the key changes involved adding signals and bindings for wrap-around links:

1. Signal Declaration: New `sc_signal` pairs (packet and ack) were declared specifically for the torus wrap-around connections . These signals connect the North ports of the top row (routers 0-3) to the South ports of the bottom row (routers 12-15) and vice-versa, and similarly connect the West ports of the left column (0, 4, 8, 12) to the East ports of the right column (3, 7, 11, 15) and vice-versa.

2. Port Binding: Additional port binding statements were added to connect these new wrap-around signals to the corresponding North/South and East/West ports of the edge routers.

IV. SIMULATOR DESIGN AND ARCHITECTURE

Synthesizing the modifications, the design of the enhanced NoC simulator can be characterized by its architecture, communication flow, and key features.

A. Architecture:

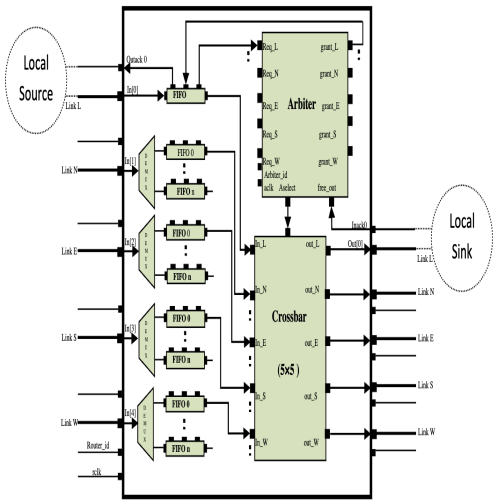

Figure 8: 5x5 Generic Router

Fig. 1. The block diagram of the old router referenced in the the report

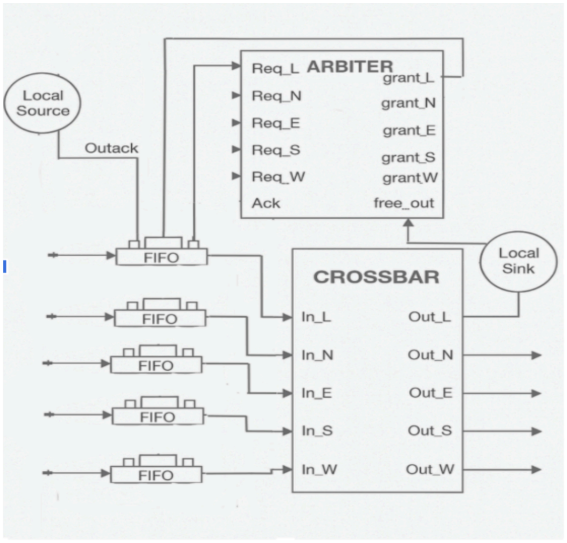

Fig. 2. The block diagram of the router used in this project

1) 4x4 Mesh NoC: The simulator implements a 16-node Network-on-Chip arranged in a 4x4 mesh topology. Each node comprises a source, sink, and router module. Routers interconnect with neighbors (N, S, E, W where applicable), and edge/corner ports are terminated. Fig. 1 illustrates this structure.

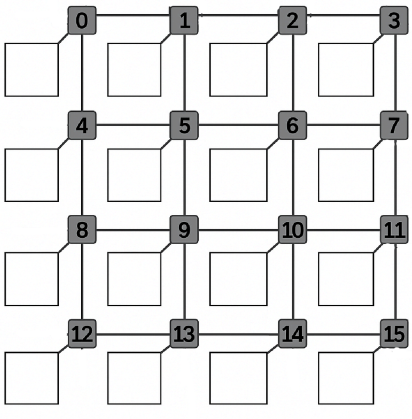

Fig. 3. The structure of the 4x4 Mesh used in this project

2) 4x4 Torus NoC:The simulator implements a 16-node (4x4) Torus topology. Each node consists of a source, sink, and router. Routers connect to their North, South, East, and West neighbors. Crucially, wrap-around connections link edge routers: North ports of row 0 connect to South ports of row 3 (and vice-versa), and West ports of column 0 connect to East ports of column 3 (and vice-versa).

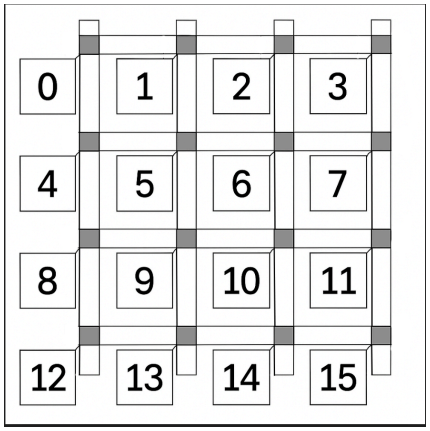

Fig. 4. The structure of the 4x4 Torus used in this project

B. Communication Flow

Communication begins with interactive configuration, followed by sc_main initialization (ID assignment, connection binding, source enabling). Enabled sources inject 5-flit packets (header with destination, body/tail with dest=-1) into their local router when downstream buffer space is available. Flits arriving at a router enter a FIFO, which requests arbitration. The arbiter processes the request: if it's a header, it performs XY routing, checks resource availability, grants access, and updates wormhole state; if it's a body/tail flit, it follows the established path. Upon grant, the FIFO sends the flit to the crossbar, which switches it to the designated output port towards the next hop or the local sink. At the destination sink, flits are received, counted, and

acknowledged back to the router, implementing basic flow control. The tail flit triggers connection teardown in the arbiter.

C. Key Design Features

The design employs a 4x4 Mesh and 4x4 Torus topology with deterministic XY Dimension-Order Routing. It uses Wormhole switching with flit-level acknowledgment for flow control. Configuration is interactive, allowing user-defined traffic patterns, although traffic generation per source is currently limited to one 5-flit packet. The design retains modularity and incorporates separate clock domains for sources versus the router/sink network. The 4x4 Torus topology features routing which is a deterministic shortest-path XY DOR adapted for torus.

V. DESIGN ANALYSIS

The modifications represent a substantial upgrade but warrant critical analysis regarding strengths and weaknesses.

A. Strengths and Advantages

The enhanced simulator demonstrates notable strengths in scalability, representativeness, and flexibility. Migrating to the 4x4 mesh and 4x4 torus provides a more relevant platform for NoC research compared to the original 1x2 setup. Interactive configuration allows users to easily define diverse communication scenarios without recompiling module code, accelerating experimentation. Crucially, the refined arbiter logic correctly implements the stateful nature of wormhole switching using a standard, deadlock-free XY routing algorithm, addressing a key project requirement. Furthermore, code structure improvements like enhanced debuggability (via `printf` and tracing) contribute positively to maintainability and verification efforts. The primary advantage of additionally implementing the torus is the potential for improved performance due to shorter average path lengths compared to the mesh, enabled by the wrap-around links. This can lead to lower latency for many communication pairs.

B. Weaknesses, Issues, and Potential Problems

From a performance modeling perspective, the lack of Virtual Channels makes the simulator highly susceptible to Head-of-Line blocking, limiting its accuracy under congestion. The basic acknowledgment-based flow control, simplified traffic generation (single 5-flit packet per source), fixed-priority arbitration implied by the arbiter's sequential processing is further limitation for realistic performance studies.
The most significant new concern introduced by the torus topology is the increased potential for routing deadlock. While the shortest-path XY algorithm itself is deterministic, applying it directly on a torus with wormhole switching and without additional mechanisms like VCs can lead to deadlock scenarios due to the cyclic dependencies created by the wrap-around links.

C. Addressing Project Requirements

The implemented modifications directly address several project requirements from the original description. Arbiter wormhole logic is addressed by

the revised arbiter.cpp logic, contingent on fixing the req signal issue. Requirement of 4x4 mesh design is substantially fulfilled by the main_noc.cpp implementation, which instantiates the mesh and allows testing via interactive configuration. The implementation of the torus topology and associated routing logic directly addresses the bonus requirement from the project description. It demonstrates the capability to model and simulate this alternative topology.

VII. Results

```
Enter the number of source-destination exchanges to configure (1-16): 15

--- Configuring Exchange #1 ---
Enter Source ID (0-15): 0
Enter Destination Sink ID (0-15): 1
Configured: Source 0 -> Sink 1

--- Configuring Exchange #2 ---
Enter Source ID (0-15): 1
Enter Destination Sink ID (0-15): 2
Configured: Source 1 -> Sink 2

--- Configuring Exchange #3 ---
Enter Source ID (0-15): 2
Enter Destination Sink ID (0-15): 3
Configured: Source 2 -> Sink 3

--- Configuring Exchange #4 ---
Enter Source ID (0-15): 3
Enter Destination Sink ID (0-15): 7
Configured: Source 3 -> Sink 7

--- Configuring Exchange #5 ---
Enter Source ID (0-15):
7
Enter Destination Sink ID (0-15): 6
Configured: Source 7 -> Sink 6

--- Configuring Exchange #6 ---
Enter Source ID (0-15): 6
Enter Destination Sink ID (0-15): 5
Configured: Source 6 -> Sink 5

--- Configuring Exchange #7 ---
Enter Source ID (0-15): 5
Enter Destination Sink ID (0-15): 4
Configured: Source 5 -> Sink 4

--- Configuring Exchange #8 ---
Enter Source ID (0-15): 4
Enter Destination Sink ID (0-15): 8
Configured: Source 4 -> Sink 8

--- Configuring Exchange #9 ---
Enter Source ID (0-15): 8
Enter Destination Sink ID (0-15): 9
Configured: Source 8 -> Sink 9

--- Configuring Exchange #10 ---
Enter Source ID (0-15): 9
Enter Destination Sink ID (0-15): 10
Configured: Source 9 -> Sink 10

--- Configuring Exchange #11 ---
Enter Source ID (0-15): 10
Enter Destination Sink ID (0-15): 11
Configured: Source 10 -> Sink 11
```

Fig. 5. Interactive configuration of the simulation by selecting unique source-sink pairs.

```
End of Simulation...

Configured Exchanges:
  Source 0 -> Sink 1
  Source 1 -> Sink 2
  Source 2 -> Sink 3
  Source 3 -> Sink 7
  Source 7 -> Sink 6
  Source 6 -> Sink 5
  Source 5 -> Sink 4
  Source 4 -> Sink 8
  Source 8 -> Sink 9
  Source 9 -> Sink 10
  Source 10 -> Sink 11
  Source 11 -> Sink 15
  Source 15 -> Sink 14
  Source 14 -> Sink 13
  Source 13 -> Sink 12

Total number of packets sent by enabled sources: 75
Total number of packets received by targeted sinks: 75
  Press "Return" key to end the program...
```

Fig. 6. Confirming that the simulation ran successfully, delivering all packets across the 4x4 torus/mesh Noc.

The output confirms that router, source, sink, and connection logic are functioning correctly.

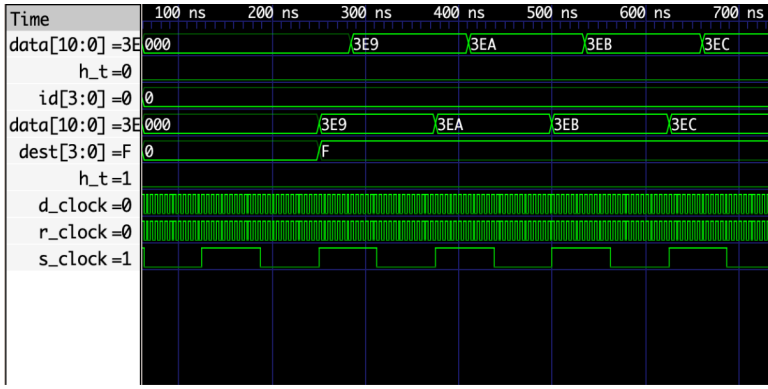

Fig. 7. Waveform of the 4x4 mesh router showcasing packet exchanges.

Source 0 ejects a 5-flit packet destined for Sink 15. The header flit is shown traversing the Mesh network and is inserted into Sink 15, experiencing a network latency of 36 ns. The subsequent flits follow this path, completing the packet reception at the sink.

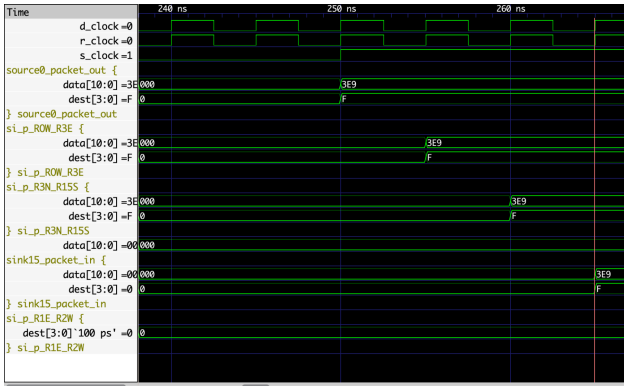

Fig. 8. This waveform snapshot illustrates a packet transmission within a Torus topology

source0_packet_out: Represents the flit currently being output by Source module 0.si_p_<From>_<To> (e.g., si_p_R0W_R3E): Represents the flit currently traversing the physical link *from* the specified port of the source router *to* the specified port of the destination router. The naming R0W_R3E indicates a link from Router 0's West port to Router 3's East port – a characteristic wrap-around connection in a Torus. Similarly, R3N_R15S connects Router 3 North to Router 15 South. p likely signifies 'packet' data.sink15_packet_in: Represents the flit arriving at the input of Sink module 15.

We can trace the journey of a single flit originating from Source 0 destined for Sink 15:

1. Injection (Source 0 -> Router 0)
2. Hop 1 (Router 0 -> Router 3)
3. Hop 2 (Router 3 -> Router 15)
4. Hop 3 (Router 15 -> Sink 15)

Why No Packet on si_p_R1E_R2W?

The signal si_p_R1E_R2W represents the link between Router 1 (East port) and Router 2 (West port). The packet trace we followed originated at Source 0 and took the path via Router 0 -> Router 3 -> Router 15, utilizing wrap-around links. This specific packet never traversed Router 1 or Router 2.
Transit Time (15 ns)

VII. CONCLUSION

In conclusion, the modifications detailed have successfully extended the baseline SystemC NoC simulator to a configurable 4x4 mesh and 4x4 torus topology, incorporating key features like interactive setup and a refined arbiter implementing stateful wormhole XY routing. This represents a significant advancement in the simulator's capability and relevance.

However, critical issues must be resolved to ensure functional correctness. Despite these necessary corrections, the modified design provides a solid foundation.

Appendix A

Altered code for 4x4 Mesh

Source.cpp

```
// source.cpp
#include "source.h" // Include header file for source module
#include <iostream> // For cout
// Function to generate and send packets
void source::func() {
   packet v_packet_out;          // Declare a packet variable
   v_packet_out.data = 1000;     // Initialize packet data base value
   v_packet_out.pkt_clk = '0';   // Initialize packet clock state
   // Wait for the enable signal to be potentially set by sc_main
   // This prevents sending packets before the user input is processed
   // A small initial delay might be needed depending on simulation setup timing.
   wait(1); // Wait 1 clock cycle initially
   while (true) {               // Infinite loop
      wait();                   // Wait for a positive clock edge
      // Only proceed if this specific source instance is enabled externally
      if (!enable_source.read()) {
         continue; // Skip the rest of the loop if not enabled by external control
      }
      // Check if the downstream buffer is ready (acknowledge is low)
      if (!ach_in.read()) {
         printf("Acknowledge not received in source %d\n", (int)source_id.read());
         // --- Modification Start ---
         // Check if we have already sent 5 packets
         if (pkt_snt >= 5) {
            // Optional: Maybe print a message indicating completion
            // std::cout << "Source " << source_id.read() << " finished sending 5 packets." << std::endl;
            continue; // Stop trying to send more packets from this source
         }
         // --- Modification End ---
         // Prepare the packet
         // Increment data based on source ID (ensures unique data for debugging)
         v_packet_out.data = v_packet_out.data + source_id.read() + 1;
         v_packet_out.id = source_id.read();        // Set packet source ID

         if(pkt_snt == 0) {
            v_packet_out.dest = target_dest_id.read(); // Set destination ID for the first packet
         }
         else {
            v_packet_out.dest = -1; // Keep the same destination for subsequent packets
         }


         v_packet_out.pkt_clk = ~v_packet_out.pkt_clk; // Toggle packet clock (for potential downstream logic)
         v_packet_out.h_t = false;                  // Default to body flit
```

```
            // Increment packet sent counter *after* preparing and intending to send
            pkt_snt++;
            // Mark every 5th packet as a tail flit (example wormhole logic)
            // Note: Since we stop *at* 5, the 5th packet will be the tail.
            if ((pkt_snt % 5) == 0) {
               v_packet_out.h_t = true;
            }
            // Write the packet to the output port
            packet_out.write(v_packet_out);
            // Print packet details
            std::cout << "New Pkt with data: " << v_packet_out.data
                      << " is sent by source: " << source_id.read()
                      << " (Count: " << pkt_snt << ")" // Added count for clarity
                      << " with h_t " << v_packet_out.h_t
                      << " with pcket_clk " << v_packet_out.pkt_clk
                      << "  to Destination:   " << v_packet_out.dest << std::endl;
         }
         // Note: Removed the 'goto exclode;' logic as it seemed problematic.
         // The enable_source flag now controls which source sends.
      } // End while true
   } // End source::func
```

Source.h

```
// source.h
#ifndef SOURCE_H
#define SOURCE_H
#include "systemc.h" // Include SystemC header
#include "packet.h"  // Include packet header
// Define the source module
SC_MODULE(source) {
   // --- Ports ---
   sc_out<packet> packet_out;         // Output port for packets
   sc_in<sc_uint<4> > source_id;      // Input port for this source's ID
   sc_in<bool> ach_in;                // Input port for acknowledgment from router buffer
   sc_in<sc_uint<4> > target_dest_id;  // Input port for the target destination ID
   sc_in<bool> enable_source;         // Input port to enable/disable this source
   sc_in_clk CLK;                     // Clock input port
   // --- Internal Variables ---
   int pkt_snt;                       // Variable for recording packets sent
   // --- Processes ---
   void func();                       // Main packet generation process
   // --- Constructor ---
   SC_CTOR(source) {                  // Constructor
      SC_CTHREAD(func, CLK.pos());    // Define clocked thread sensitive to positive clock edge
      pkt_snt = 0;                    // Initialize packet sent counter
   }
}; // End of SC_MODULE
#endif // SOURCE_H
```

Main_noc.cpp

```cpp
// main_noc.cpp (Interactive Version)
#include "systemc.h"
#include <iostream>
#include <vector>      // For storing exchanges
#include <utility>     // For std::pair
#include <limits>      // For numeric_limits
#include <set>         // For tracking used sources/sinks
#include <string>      // For input line
// Include project headers
#include "packet.h"
#include "source.h"
#include "sink.h"
#include "router.h" // Assuming router.h includes the necessary sub-modules
// Function to safely get integer input from user within a range
int get_validated_int(const std::string& prompt, int min_val, int max_val) {
    int value;
    while (true) {
        std::cout << prompt;
        if (std::cin >> value && value >= min_val && value <= max_val) {
            // Clear the rest of the line, in case user entered extra characters
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            return value;
        } else {
            std::cout << "Invalid input. Please enter an integer between "
                      << min_val << " and " << max_val << "." << std::endl;
            std::cin.clear(); // Clear error flags
            // Discard the invalid input
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        }
    }
}
int sc_main(int argc, char *argv[])
{
    // --- User Interaction ---
    std::cout << std::endl;
    std::cout << "-----------------------------------------------------------------------------" << std::endl;
    std::cout << " Interactive NoC Simulation Setup (Assumes 4x4 Mesh with 16 Routers)" << std::endl;
    std::cout << "-----------------------------------------------------------------------------" << std::endl;
    int num_exchanges = get_validated_int("Enter the number of source-destination exchanges to configure (1-16): ", 1, 16);
    std::vector<std::pair<int, int>> exchanges;
    std::set<int> used_sources;
    std::set<int> used_sinks;
    for (int i = 0; i < num_exchanges; ++i) {
        std::cout << "\n--- Configuring Exchange #" << (i + 1) << " ---" << std::endl;
        int src_id = -1;
        int dest_id = -1;
        // Get Source ID
        while (true) {
            src_id = get_validated_int("Enter Source ID (0-15): ", 0, 15);
            if (used_sources.find(src_id) == used_sources.end()) {
                break; // Source not used yet
            } else {
                std::cout << "Source " << src_id << " has already been assigned. Please choose another." << std::endl;
            }
        }
        // Get Destination ID
        while (true) {
```

```
        dest_id = get_validated_int("Enter Destination Sink ID (0-15): ", 0, 15);
        if (dest_id == src_id) {
            std::cout << "Destination Sink ID cannot be the same as the Source ID. Please choose another." << std::endl;
        } else if (used_sinks.find(dest_id) == used_sinks.end()) {
            break; // Sink not used yet and different from source
        } else {
            std::cout << "Sink " << dest_id << " has already been assigned. Please choose another." << std::endl;
        }
    }
    // Store valid exchange
    exchanges.push_back({src_id, dest_id});
    used_sources.insert(src_id);
    used_sinks.insert(dest_id);
    std::cout << "Configured: Source " << src_id << " -> Sink " << dest_id << std::endl;
}
// --- Signal Declarations ---
// Clocks
        sc_clock s_clock("S_CLOCK", 125, SC_NS, 0.5, 0.0, SC_NS); // Source clock
        sc_clock r_clock("R_CLOCK", 5, SC_NS, 0.5, 10.0, SC_NS);  // Router clock (assuming one clock for all routers)
        sc_clock d_clock("D_CLOCK", 5, SC_NS, 0.5, 10.0, SC_NS); // Destination clock (using same as router for simplicity)
// Signals for Sources (Arrays for 16 sources)
        sc_signal<packet> si_source[16];      // Packet output from sources
        sc_signal<bool>   si_ack_src[16];     // Ack input to sources (from router buffer)
sc_signal<bool>   enable_signals[16];  // Enable signal for sources
sc_signal<sc_uint<4>> dest_signals[16]; // Destination ID signal for sources
sc_signal<sc_uint<4>> source_ids[16];   // Source ID signals
// Signals for Sinks (Arrays for 16 sinks)
        sc_signal<packet> si_sink[16];        // Packet input to sinks
        sc_signal<bool>   si_ack_sink[16];    // Ack output from sinks
sc_signal<sc_uint<4>> sink_ids[16];     // Sink ID signals
// Signals for Router Interconnects (Simplified - focus is on source/sink setup)
        sc_signal<packet> si_input[16]; // Placeholder for router inputs from other routers
        sc_signal<packet> si_output[16]; // Placeholder for router outputs to other routers
sc_signal<bool>   si_ack_in[16]; // Placeholder
sc_signal<bool>   si_ack_ou[16]; // Placeholder
        // --- Module Instantiations ---
// Signals for Inter-Router Connections (4x4 mesh)
// Naming: si_p/a_<FromRouter>_<Direction>_<ToRouter>_<Direction>
// Based on Mapping: 0=L, 1=N, 2=E, 3=S, 4=W

// Horizontal connections (East-West)
// Row 0
sc_signal<packet> si_p_R0E_R1W, si_p_R1W_R0E; sc_signal<bool> si_a_R0E_R1W, si_a_R1W_R0E;
sc_signal<packet> si_p_R1E_R2W, si_p_R2W_R1E; sc_signal<bool> si_a_R1E_R2W, si_a_R2W_R1E;
sc_signal<packet> si_p_R2E_R3W, si_p_R3W_R2E; sc_signal<bool> si_a_R2E_R3W, si_a_R3W_R2E;
// Row 1
sc_signal<packet> si_p_R4E_R5W, si_p_R5W_R4E; sc_signal<bool> si_a_R4E_R5W, si_a_R5W_R4E;
sc_signal<packet> si_p_R5E_R6W, si_p_R6W_R5E; sc_signal<bool> si_a_R5E_R6W, si_a_R6W_R5E;
sc_signal<packet> si_p_R6E_R7W, si_p_R7W_R6E; sc_signal<bool> si_a_R6E_R7W, si_a_R7W_R6E;
// Row 2
sc_signal<packet> si_p_R8E_R9W, si_p_R9W_R8E; sc_signal<bool> si_a_R8E_R9W, si_a_R9W_R8E;
sc_signal<packet> si_p_R9E_R10W, si_p_R10W_R9E; sc_signal<bool> si_a_R9E_R10W, si_a_R10W_R9E;
sc_signal<packet> si_p_R10E_R11W, si_p_R11W_R10E; sc_signal<bool> si_a_R10E_R11W, si_a_R11W_R10E;
// Row 3
sc_signal<packet> si_p_R12E_R13W, si_p_R13W_R12E; sc_signal<bool> si_a_R12E_R13W, si_a_R13W_R12E;
sc_signal<packet> si_p_R13E_R14W, si_p_R14W_R13E; sc_signal<bool> si_a_R13E_R14W, si_a_R14W_R13E;
sc_signal<packet> si_p_R14E_R15W, si_p_R15W_R14E; sc_signal<bool> si_a_R14E_R15W, si_a_R15W_R14E;
```

```
// Vertical connections (North-South)
// Column 0
sc_signal<packet> si_p_R0S_R4N, si_p_R4N_R0S; sc_signal<bool> si_a_R0S_R4N, si_a_R4N_R0S;
sc_signal<packet> si_p_R4S_R8N, si_p_R8N_R4S; sc_signal<bool> si_a_R4S_R8N, si_a_R8N_R4S;
sc_signal<packet> si_p_R8S_R12N, si_p_R12N_R8S; sc_signal<bool> si_a_R8S_R12N, si_a_R12N_R8S;
// Column 1
sc_signal<packet> si_p_R1S_R5N, si_p_R5N_R1S; sc_signal<bool> si_a_R1S_R5N, si_a_R5N_R1S;
sc_signal<packet> si_p_R5S_R9N, si_p_R9N_R5S; sc_signal<bool> si_a_R5S_R9N, si_a_R9N_R5S;
sc_signal<packet> si_p_R9S_R13N, si_p_R13N_R9S; sc_signal<bool> si_a_R9S_R13N, si_a_R13N_R9S;
// Column 2
sc_signal<packet> si_p_R2S_R6N, si_p_R6N_R2S; sc_signal<bool> si_a_R2S_R6N, si_a_R6N_R2S;
sc_signal<packet> si_p_R6S_R10N, si_p_R10N_R6S; sc_signal<bool> si_a_R6S_R10N, si_a_R10N_R6S;
sc_signal<packet> si_p_R10S_R14N, si_p_R14N_R10S; sc_signal<bool> si_a_R10S_R14N, si_a_R14N_R10S;
// Column 3
sc_signal<packet> si_p_R3S_R7N, si_p_R7N_R3S; sc_signal<bool> si_a_R3S_R7N, si_a_R7N_R3S;
sc_signal<packet> si_p_R7S_R11N, si_p_R11N_R7S; sc_signal<bool> si_a_R7S_R11N, si_a_R11N_R7S;
sc_signal<packet> si_p_R11S_R15N, si_p_R15N_R11S; sc_signal<bool> si_a_R11S_R15N, si_a_R15N_R11S;

// *** Unique Dummy signals for unconnected edge ports ***
// Top row routers (North edge)
// Router 0 (Top-Left corner - Edges: North(1), West(4))
sc_signal<packet> dummy_p_r0_i1, dummy_p_r0_o1, dummy_p_r0_i4, dummy_p_r0_o4;
sc_signal<bool> dummy_a_r0_i1, dummy_a_r0_o1, dummy_a_r0_i4, dummy_a_r0_o4;
// Router 1 (Top row - Edges: North(1))
sc_signal<packet> dummy_p_r1_i1, dummy_p_r1_o1;
sc_signal<bool> dummy_a_r1_i1, dummy_a_r1_o1;
// Router 2 (Top row - Edges: North(1))
sc_signal<packet> dummy_p_r2_i1, dummy_p_r2_o1;
sc_signal<bool> dummy_a_r2_i1, dummy_a_r2_o1;
// Router 3 (Top-Right corner - Edges: North(1), East(2))
sc_signal<packet> dummy_p_r3_i1, dummy_p_r3_o1, dummy_p_r3_i2, dummy_p_r3_o2;
sc_signal<bool> dummy_a_r3_i1, dummy_a_r3_o1, dummy_a_r3_i2, dummy_a_r3_o2;

// Left column routers (West edge, excluding corners)
// Router 4 (Left column - Edges: West(4))
sc_signal<packet> dummy_p_r4_i4, dummy_p_r4_o4;
sc_signal<bool> dummy_a_r4_i4, dummy_a_r4_o4;
// Router 8 (Left column - Edges: West(4))
sc_signal<packet> dummy_p_r8_i4, dummy_p_r8_o4;
sc_signal<bool> dummy_a_r8_i4, dummy_a_r8_o4;

// Right column routers (East edge, excluding corners)
// Router 7 (Right column - Edges: East(2))
sc_signal<packet> dummy_p_r7_i2, dummy_p_r7_o2;
sc_signal<bool> dummy_a_r7_i2, dummy_a_r7_o2;
// Router 11 (Right column - Edges: East(2))
sc_signal<packet> dummy_p_r11_i2, dummy_p_r11_o2;
sc_signal<bool> dummy_a_r11_i2, dummy_a_r11_o2;

// Bottom row routers (South edge)
// Router 12 (Bottom-Left corner - Edges: South(3), West(4))
sc_signal<packet> dummy_p_r12_i3, dummy_p_r12_o3, dummy_p_r12_i4, dummy_p_r12_o4;
sc_signal<bool> dummy_a_r12_i3, dummy_a_r12_o3, dummy_a_r12_i4, dummy_a_r12_o4;
// Router 13 (Bottom row - Edges: South(3))
sc_signal<packet> dummy_p_r13_i3, dummy_p_r13_o3;
sc_signal<bool> dummy_a_r13_i3, dummy_a_r13_o3;
```

```
// Router 14 (Bottom row - Edges: South(3))
sc_signal<packet> dummy_p_r14_i3, dummy_p_r14_o3;
sc_signal<bool> dummy_a_r14_i3, dummy_a_r14_o3;
// Router 15 (Bottom-Right corner - Edges: South(3), East(2))
sc_signal<packet> dummy_p_r15_i3, dummy_p_r15_o3, dummy_p_r15_i2, dummy_p_r15_o2;
sc_signal<bool> dummy_a_r15_i3, dummy_a_r15_o3, dummy_a_r15_i2, dummy_a_r15_o2;

// Interior routers (Local ports for routers without sources/sinks)
sc_signal<packet> dummy_p_r4_i0, dummy_p_r4_o0, dummy_p_r5_i0, dummy_p_r5_o0;
sc_signal<bool> dummy_a_r4_i0, dummy_a_r4_o0, dummy_a_r5_i0, dummy_a_r5_o0;
sc_signal<packet> dummy_p_r6_i0, dummy_p_r6_o0, dummy_p_r7_i0, dummy_p_r7_o0;
sc_signal<bool> dummy_a_r6_i0, dummy_a_r6_o0, dummy_a_r7_i0, dummy_a_r7_o0;
sc_signal<packet> dummy_p_r8_i0, dummy_p_r8_o0, dummy_p_r9_i0, dummy_p_r9_o0;
sc_signal<bool> dummy_a_r8_i0, dummy_a_r8_o0, dummy_a_r9_i0, dummy_a_r9_o0;
sc_signal<packet> dummy_p_r10_i0, dummy_p_r10_o0, dummy_p_r11_i0, dummy_p_r11_o0;
sc_signal<bool> dummy_a_r10_i0, dummy_a_r10_o0, dummy_a_r11_i0, dummy_a_r11_o0;
sc_signal<packet> dummy_p_r12_i0, dummy_p_r12_o0, dummy_p_r13_i0, dummy_p_r13_o0;
sc_signal<bool> dummy_a_r12_i0, dummy_a_r12_o0, dummy_a_r13_i0, dummy_a_r13_o0;
sc_signal<packet> dummy_p_r14_i0, dummy_p_r14_o0, dummy_p_r15_i0, dummy_p_r15_o0;
sc_signal<bool> dummy_a_r14_i0, dummy_a_r14_o0, dummy_a_r15_i0, dummy_a_r15_o0;
// Instantiate Sources (0 to 3 only, as per original implementation)
source* sources[16];
for (int i = 0; i < 16; ++i) {
    std::string name = "source" + std::to_string(i);
    sources[i] = new source(name.c_str());
    sources[i]->packet_out(si_source[i]);
    sources[i]->source_id(source_ids[i]);
    sources[i]->ach_in(si_ack_src[i]);
    sources[i]->CLK(s_clock);
    sources[i]->enable_source(enable_signals[i]);     // Connect enable port
    sources[i]->target_dest_id(dest_signals[i]); // Connect destination port
}
// Instantiate Sinks (0 to 15 only, as per original implementation)
sink* sinks[16];
for (int i = 0; i < 16; ++i) {
    std::string name = "sink" + std::to_string(i);
    sinks[i] = new sink(name.c_str());
    sinks[i]->packet_in(si_sink[i]);
    sinks[i]->ack_out(si_ack_sink[i]);
    sinks[i]->sink_id(sink_ids[i]);
    sinks[i]->sclk(d_clock); // Using destination clock
}
// Instantiate Routers (0 to 15) - 4x4 Mesh
router* routers[16];
for (int i = 0; i < 16; ++i) {
    std::string name = "router" + std::to_string(i);
    routers[i] = new router(name.c_str());
    // Connect Clock for all routers
    routers[i]->rclk(r_clock);

    // Connect Router ID - For now, using modulo 16 to match with available sources/sinks
    routers[i]->router_id(source_ids[i % 16]);
}

// Connect sources and sinks only to the first 16 routers (as per original implementation)
for (int i = 0; i < 16; ++i) {
    // Connect local source/sink
```

```
    routers[i]->in0(si_source[i]);      // Input from local source
    routers[i]->outack0(si_ack_src[i]); // Ack back to local source
    routers[i]->out0(si_sink[i]);       // Output to local sink
    routers[i]->inack0(si_ack_sink[i]); // Ack from local sink
}
// --- Configuration before Simulation ---
// Assign IDs
for (int i = 0; i < 16; ++i) {
    source_ids[i].write(i);
    sink_ids[i].write(i);
    // Initialize control signals
    enable_signals[i].write(false); // Disable all sources initially
    dest_signals[i].write(0);       // Default destination
}
// Enable and configure sources based on user input
for (const auto& exchange : exchanges) {
    int src = exchange.first;
    int dest = exchange.second;
    enable_signals[src].write(true);
    dest_signals[src].write(dest);
}
// Connect all routers in the 4x4 mesh
// Horizontal connections (East-West)
// Row 0
routers[0]->out2(si_p_R0E_R1W);    routers[0]->inack2(si_a_R1W_R0E);
routers[1]->in4(si_p_R0E_R1W);     routers[1]->outack4(si_a_R1W_R0E);
routers[1]->out4(si_p_R1W_R0E);    routers[1]->inack4(si_a_R0E_R1W);
routers[0]->in2(si_p_R1W_R0E);     routers[0]->outack2(si_a_R0E_R1W);

routers[1]->out2(si_p_R1E_R2W);    routers[1]->inack2(si_a_R2W_R1E);
routers[2]->in4(si_p_R1E_R2W);     routers[2]->outack4(si_a_R2W_R1E);
routers[2]->out4(si_p_R2W_R1E);    routers[2]->inack4(si_a_R1E_R2W);
routers[1]->in2(si_p_R2W_R1E);     routers[1]->outack2(si_a_R1E_R2W);

routers[2]->out2(si_p_R2E_R3W);    routers[2]->inack2(si_a_R3W_R2E);
routers[3]->in4(si_p_R2E_R3W);     routers[3]->outack4(si_a_R3W_R2E);
routers[3]->out4(si_p_R3W_R2E);    routers[3]->inack4(si_a_R2E_R3W);
routers[2]->in2(si_p_R3W_R2E);     routers[2]->outack2(si_a_R2E_R3W);

// Row 1
routers[4]->out2(si_p_R4E_R5W);    routers[4]->inack2(si_a_R5W_R4E);
routers[5]->in4(si_p_R4E_R5W);     routers[5]->outack4(si_a_R5W_R4E);
routers[5]->out4(si_p_R5W_R4E);    routers[5]->inack4(si_a_R4E_R5W);
routers[4]->in2(si_p_R5W_R4E);     routers[4]->outack2(si_a_R4E_R5W);

routers[5]->out2(si_p_R5E_R6W);    routers[5]->inack2(si_a_R6W_R5E);
routers[6]->in4(si_p_R5E_R6W);     routers[6]->outack4(si_a_R6W_R5E);
routers[6]->out4(si_p_R6W_R5E);    routers[6]->inack4(si_a_R5E_R6W);
routers[5]->in2(si_p_R6W_R5E);     routers[5]->outack2(si_a_R5E_R6W);

routers[6]->out2(si_p_R6E_R7W);    routers[6]->inack2(si_a_R7W_R6E);
routers[7]->in4(si_p_R6E_R7W);     routers[7]->outack4(si_a_R7W_R6E);
routers[7]->out4(si_p_R7W_R6E);    routers[7]->inack4(si_a_R6E_R7W);
routers[6]->in2(si_p_R7W_R6E);     routers[6]->outack2(si_a_R6E_R7W);

// Row 2
routers[8]->out2(si_p_R8E_R9W);    routers[8]->inack2(si_a_R9W_R8E);
```

```
routers[9]->in4(si_p_R8E_R9W);      routers[9]->outack4(si_a_R9W_R8E);
routers[9]->out4(si_p_R9W_R8E);     routers[9]->inack4(si_a_R8E_R9W);
routers[8]->in2(si_p_R9W_R8E);      routers[8]->outack2(si_a_R8E_R9W);

routers[9]->out2(si_p_R9E_R10W);    routers[9]->inack2(si_a_R10W_R9E);
routers[10]->in4(si_p_R9E_R10W);    routers[10]->outack4(si_a_R10W_R9E);
routers[10]->out4(si_p_R10W_R9E);   routers[10]->inack4(si_a_R9E_R10W);
routers[9]->in2(si_p_R10W_R9E);     routers[9]->outack2(si_a_R9E_R10W);

routers[10]->out2(si_p_R10E_R11W);  routers[10]->inack2(si_a_R11W_R10E);
routers[11]->in4(si_p_R10E_R11W);   routers[11]->outack4(si_a_R11W_R10E);
routers[11]->out4(si_p_R11W_R10E);  routers[11]->inack4(si_a_R10E_R11W);
routers[10]->in2(si_p_R11W_R10E);   routers[10]->outack2(si_a_R10E_R11W);

// Row 3
routers[12]->out2(si_p_R12E_R13W);  routers[12]->inack2(si_a_R13W_R12E);
routers[13]->in4(si_p_R12E_R13W);   routers[13]->outack4(si_a_R13W_R12E);
routers[13]->out4(si_p_R13W_R12E);  routers[13]->inack4(si_a_R12E_R13W);
routers[12]->in2(si_p_R13W_R12E);   routers[12]->outack2(si_a_R12E_R13W);

routers[13]->out2(si_p_R13E_R14W);  routers[13]->inack2(si_a_R14W_R13E);
routers[14]->in4(si_p_R13E_R14W);   routers[14]->outack4(si_a_R14W_R13E);
routers[14]->out4(si_p_R14W_R13E);  routers[14]->inack4(si_a_R13E_R14W);
routers[13]->in2(si_p_R14W_R13E);   routers[13]->outack2(si_a_R13E_R14W);

routers[14]->out2(si_p_R14E_R15W);  routers[14]->inack2(si_a_R15W_R14E);
routers[15]->in4(si_p_R14E_R15W);   routers[15]->outack4(si_a_R15W_R14E);
routers[15]->out4(si_p_R15W_R14E);  routers[15]->inack4(si_a_R14E_R15W);
routers[14]->in2(si_p_R15W_R14E);   routers[14]->outack2(si_a_R14E_R15W);
// Vertical connections (North-South)
// Column 0
routers[0]->out3(si_p_R0S_R4N);     routers[0]->inack3(si_a_R4N_R0S);
routers[4]->in1(si_p_R0S_R4N);      routers[4]->outack1(si_a_R4N_R0S);
routers[4]->out1(si_p_R4N_R0S);     routers[4]->inack1(si_a_R0S_R4N);
routers[0]->in3(si_p_R4N_R0S);      routers[0]->outack3(si_a_R0S_R4N);

routers[4]->out3(si_p_R4S_R8N);     routers[4]->inack3(si_a_R8N_R4S);
routers[8]->in1(si_p_R4S_R8N);      routers[8]->outack1(si_a_R8N_R4S);
routers[8]->out1(si_p_R8N_R4S);     routers[8]->inack1(si_a_R4S_R8N);
routers[4]->in3(si_p_R8N_R4S);      routers[4]->outack3(si_a_R4S_R8N);

routers[8]->out3(si_p_R8S_R12N);    routers[8]->inack3(si_a_R12N_R8S);
routers[12]->in1(si_p_R8S_R12N);    routers[12]->outack1(si_a_R12N_R8S);
routers[12]->out1(si_p_R12N_R8S);   routers[12]->inack1(si_a_R8S_R12N);
routers[8]->in3(si_p_R12N_R8S);     routers[8]->outack3(si_a_R8S_R12N);

// Column 1
routers[1]->out3(si_p_R1S_R5N);     routers[1]->inack3(si_a_R5N_R1S);
routers[5]->in1(si_p_R1S_R5N);      routers[5]->outack1(si_a_R5N_R1S);
routers[5]->out1(si_p_R5N_R1S);     routers[5]->inack1(si_a_R1S_R5N);
routers[1]->in3(si_p_R5N_R1S);      routers[1]->outack3(si_a_R1S_R5N);

routers[5]->out3(si_p_R5S_R9N);     routers[5]->inack3(si_a_R9N_R5S);
routers[9]->in1(si_p_R5S_R9N);      routers[9]->outack1(si_a_R9N_R5S);
routers[9]->out1(si_p_R9N_R5S);     routers[9]->inack1(si_a_R5S_R9N);
routers[5]->in3(si_p_R9N_R5S);      routers[5]->outack3(si_a_R5S_R9N);
```

```
routers[9]->out3(si_p_R9S_R13N);    routers[9]->inack3(si_a_R13N_R9S);
routers[13]->in1(si_p_R9S_R13N);    routers[13]->outack1(si_a_R13N_R9S);
routers[13]->out1(si_p_R13N_R9S);   routers[13]->inack1(si_a_R9S_R13N);
routers[9]->in3(si_p_R13N_R9S);     routers[9]->outack3(si_a_R9S_R13N);

// Column 2
routers[2]->out3(si_p_R2S_R6N);     routers[2]->inack3(si_a_R6N_R2S);
routers[6]->in1(si_p_R2S_R6N);      routers[6]->outack1(si_a_R6N_R2S);
routers[6]->out1(si_p_R6N_R2S);     routers[6]->inack1(si_a_R2S_R6N);
routers[2]->in3(si_p_R6N_R2S);      routers[2]->outack3(si_a_R2S_R6N);

routers[6]->out3(si_p_R6S_R10N);    routers[6]->inack3(si_a_R10N_R6S);
routers[10]->in1(si_p_R6S_R10N);    routers[10]->outack1(si_a_R10N_R6S);
routers[10]->out1(si_p_R10N_R6S);   routers[10]->inack1(si_a_R6S_R10N);
routers[6]->in3(si_p_R10N_R6S);     routers[6]->outack3(si_a_R6S_R10N);

routers[10]->out3(si_p_R10S_R14N);  routers[10]->inack3(si_a_R14N_R10S);
routers[14]->in1(si_p_R10S_R14N);   routers[14]->outack1(si_a_R14N_R10S);
routers[14]->out1(si_p_R14N_R10S);  routers[14]->inack1(si_a_R10S_R14N);
routers[10]->in3(si_p_R14N_R10S);   routers[10]->outack3(si_a_R10S_R14N);

// Column 3
routers[3]->out3(si_p_R3S_R7N);     routers[3]->inack3(si_a_R7N_R3S);
routers[7]->in1(si_p_R3S_R7N);      routers[7]->outack1(si_a_R7N_R3S);
routers[7]->out1(si_p_R7N_R3S);     routers[7]->inack1(si_a_R3S_R7N);
routers[3]->in3(si_p_R7N_R3S);      routers[3]->outack3(si_a_R3S_R7N);

routers[7]->out3(si_p_R7S_R11N);    routers[7]->inack3(si_a_R11N_R7S);
routers[11]->in1(si_p_R7S_R11N);    routers[11]->outack1(si_a_R11N_R7S);
routers[11]->out1(si_p_R11N_R7S);   routers[11]->inack1(si_a_R7S_R11N);
routers[7]->in3(si_p_R11N_R7S);     routers[7]->outack3(si_a_R7S_R11N);

routers[11]->out3(si_p_R11S_R15N);  routers[11]->inack3(si_a_R15N_R11S);
routers[15]->in1(si_p_R11S_R15N);   routers[15]->outack1(si_a_R15N_R11S);
routers[15]->out1(si_p_R15N_R11S);  routers[15]->inack1(si_a_R11S_R15N);
routers[11]->in3(si_p_R15N_R11S);   routers[11]->outack3(si_a_R11S_R15N);
printf("Inter-Router Ports Connected\n");

// Connect Edge Ports to Dummies
// Top row routers (North edge)
// Router 0 (Top-Left corner - Edges: North(1), West(4))
routers[0]->in1(dummy_p_r0_i1);     routers[0]->outack1(dummy_a_r0_o1);
routers[0]->out1(dummy_p_r0_o1);    routers[0]->inack1(dummy_a_r0_i1);
routers[0]->in4(dummy_p_r0_i4);     routers[0]->outack4(dummy_a_r0_o4);
routers[0]->out4(dummy_p_r0_o4);    routers[0]->inack4(dummy_a_r0_i4);

// Router 1 (Top row - Edges: North(1))
routers[1]->in1(dummy_p_r1_i1);     routers[1]->outack1(dummy_a_r1_o1);
routers[1]->out1(dummy_p_r1_o1);    routers[1]->inack1(dummy_a_r1_i1);

// Router 2 (Top row - Edges: North(1))
routers[2]->in1(dummy_p_r2_i1);     routers[2]->outack1(dummy_a_r2_o1);
routers[2]->out1(dummy_p_r2_o1);    routers[2]->inack1(dummy_a_r2_i1);

// Router 3 (Top-Right corner - Edges: North(1), East(2))
routers[3]->in1(dummy_p_r3_i1);     routers[3]->outack1(dummy_a_r3_o1);
routers[3]->out1(dummy_p_r3_o1);    routers[3]->inack1(dummy_a_r3_i1);
```

```
routers[3]->in2(dummy_p_r3_i2);     routers[3]->outack2(dummy_a_r3_o2);
routers[3]->out2(dummy_p_r3_o2);    routers[3]->inack2(dummy_a_r3_i2);

// Left column routers (West edge, excluding corners)
// Router 4 (Left column - Edges: West(4))
routers[4]->in4(dummy_p_r4_i4);     routers[4]->outack4(dummy_a_r4_o4);
routers[4]->out4(dummy_p_r4_o4);    routers[4]->inack4(dummy_a_r4_i4);

// Router 8 (Left column - Edges: West(4))
routers[8]->in4(dummy_p_r8_i4);     routers[8]->outack4(dummy_a_r8_o4);
routers[8]->out4(dummy_p_r8_o4);    routers[8]->inack4(dummy_a_r8_i4);

// Right column routers (East edge, excluding corners)
// Router 7 (Right column - Edges: East(2))
routers[7]->in2(dummy_p_r7_i2);     routers[7]->outack2(dummy_a_r7_o2);
routers[7]->out2(dummy_p_r7_o2);    routers[7]->inack2(dummy_a_r7_i2);

// Router 11 (Right column - Edges: East(2))
routers[11]->in2(dummy_p_r11_i2);   routers[11]->outack2(dummy_a_r11_o2);
routers[11]->out2(dummy_p_r11_o2);  routers[11]->inack2(dummy_a_r11_i2);

// Bottom row routers (South edge)
// Router 12 (Bottom-Left corner - Edges: South(3), West(4))
routers[12]->in3(dummy_p_r12_i3);   routers[12]->outack3(dummy_a_r12_o3);
routers[12]->out3(dummy_p_r12_o3);  routers[12]->inack3(dummy_a_r12_i3);
routers[12]->in4(dummy_p_r12_i4);   routers[12]->outack4(dummy_a_r12_o4);
routers[12]->out4(dummy_p_r12_o4);  routers[12]->inack4(dummy_a_r12_i4);

// Router 13 (Bottom row - Edges: South(3))
routers[13]->in3(dummy_p_r13_i3);   routers[13]->outack3(dummy_a_r13_o3);
routers[13]->out3(dummy_p_r13_o3);  routers[13]->inack3(dummy_a_r13_i3);

// Router 14 (Bottom row - Edges: South(3))
routers[14]->in3(dummy_p_r14_i3);   routers[14]->outack3(dummy_a_r14_o3);
routers[14]->out3(dummy_p_r14_o3);  routers[14]->inack3(dummy_a_r14_i3);

// Router 15 (Bottom-Right corner - Edges: South(3), East(2))
routers[15]->in3(dummy_p_r15_i3);   routers[15]->outack3(dummy_a_r15_o3);
routers[15]->out3(dummy_p_r15_o3);  routers[15]->inack3(dummy_a_r15_i3);
routers[15]->in2(dummy_p_r15_i2);   routers[15]->outack2(dummy_a_r15_o2);
routers[15]->out2(dummy_p_r15_o2);  routers[15]->inack2(dummy_a_r15_i2);
printf("Edge Ports Connected to Dummies\n");
printf("All Module ports connected\n");
// --- Tracing (Optional) ---
        sc_trace_file *tf = sc_create_vcd_trace_file("interactive_noc_trace");
        if (tf) {
   sc_trace(tf, s_clock, "s_clock");
   sc_trace(tf, r_clock, "r_clock");
   sc_trace(tf, d_clock, "d_clock");
   for (int i = 0; i < 16; ++i) {
      sc_trace(tf, si_source[i], "source" + std::to_string(i) + "_packet_out");
      sc_trace(tf, si_sink[i], "sink" + std::to_string(i) + "_packet_in");
      sc_trace(tf, si_ack_src[i], "source" + std::to_string(i) + "_ack_in");
      sc_trace(tf, si_ack_sink[i], "sink" + std::to_string(i) + "_ack_out");
      sc_trace(tf, enable_signals[i], "source" + std::to_string(i) + "_enable");
      sc_trace(tf, dest_signals[i], "source" + std::to_string(i) + "_dest_id");
   }
```

```
    // Add tracing for router signals if needed
} else {
    std::cerr << "Warning: Could not create VCD trace file." << std::endl;
}
// --- Simulation Start ---
            std::cout << "\nStarting simulation..." << std::endl;
            std::cout << "Press \"Return\" key to start the simulation..." << std::endl;
// Clear the input buffer before getchar
std::cin.sync(); // Or other appropriate method depending on OS/compiler
            getchar();
            sc_start(10*125 + 124, SC_NS);  // Run for a specific duration
if (tf) {
                sc_close_vcd_trace_file(tf);
}
// --- Simulation End & Results ---
            std::cout << std::endl << std::endl << "--------------------------------------------------------------------------------" << std::endl;
            std::cout << "End of Simulation..." << std::endl;
// Display configured exchanges
std::cout << "\nConfigured Exchanges:" << std::endl;
if (exchanges.empty()) {
    std::cout << "  None" << std::endl;
} else {
    for (const auto& exchange : exchanges) {
        std::cout << "  Source " << exchange.first << " -> Sink " << exchange.second << std::endl;
    }
}
// Display packet counts (summing from active sources/sinks)
long long total_sent = 0;
long long total_recv = 0;
for (int i = 0; i < 16; ++i) {
    if (enable_signals[i].read()) { // Check if the source was enabled
        total_sent += sources[i]->pkt_snt;
    }
    // Check if the sink was a target in any exchange
    bool sink_was_target = false;
    for (const auto& exchange : exchanges) {
        if (exchange.second == i) {
            sink_was_target = true;
            break;
        }
    }
    if (sink_was_target) {
         total_recv += sinks[i]->pkt_recv;
    }
}
            std::cout << "\nTotal number of packets sent by enabled sources: " << total_sent << std::endl;
            std::cout << "Total number of packets received by targeted sinks: " << total_recv << std::endl;
            std::cout << "--------------------------------------------------------------------------------" << std::endl;
std::cout << "  Press \"Return\" key to end the program..." << std::endl << std::endl;
// Clear the input buffer before getchar
std::cin.sync();
            getchar();
// Clean up dynamically allocated modules
for (int i = 0; i < 16; ++i) {
    delete sources[i];
    delete sinks[i];
}
```

```
    // Clean up all routers (0-15)
    for (int i = 0; i < 16; ++i) {
        delete routers[i];
    }
    return 0;
}
```

Arbiter.cpp

```
//arbiter.cpp
#undef SC_INCLUDE_FX
#include "packet.h"    // Include packet header file
#include "arbiter.h"   // Include arbiter header file
#include <iostream>  // Include iostream for printing
// Arbiter functionality
void arbiter::func() {
  printf("Arbiter function started\n"); // Print statement
  sc_uint<1> v_connected_input[5];  // set when input is connected to an output
  sc_uint<1> v_reserved_output[6]; // set when output is reserved by a input (one
                                   // output more for simple coding)
  sc_uint<3> v_req[5];             // request signal for each input
  sc_uint<5> v_free;               // status of output in term of being free
  sc_uint<4> v_id;                 // ID of the arbiter
  sc_uint<5> v_arbit;              // arbitration result
  sc_uint<15> v_select;            // selection signal
  // Initialize variables
  for (int i = 0; i < 5; i++) {
    v_connected_input[i] = 0; // Initially no input is connected
    v_reserved_output[i] = 0;  // Initially no output is reserved
    v_req[i] = 0;              // Initially no request
  }
  v_free = 31;   // '11111' - Initially all outputs are free
  v_arbit = 0;   // Initially no output is arbitrated
  v_select = 0;  // Initially no selection

  // functionality
  while (true) { // Infinite loop
    wait();       // Wait for a clock cycle

    grant0.write(0); // reset grant for input 0
    grant1.write(0); // reset grant for input 1
    grant2.write(0); // reset grant for input 2
    grant3.write(0); // reset grant for input 3
    grant4.write(0); // reset grant for input 4
    // Update free output status
    if (!free_out0.read()) {
      v_free = v_free | 1;
    } // set the bit 0 showing the output 0 is free
    if (!free_out1.read()) {
      v_free = v_free | 2;
    }
```

```
if (!free_out2.read()) {
  v_free = v_free | 4;
}
if (!free_out3.read()) {
  v_free = v_free | 8;
}
if (!free_out4.read()) {
  v_free = v_free | 16;
}

v_id = arbiter_id.read(); // Read arbiter ID

// Process request from input 0
if (!req0.read()[4]) // if FIFO buffer is not empty
{
  printf("Request received on input 0\n"); // Print statement
  if(!v_connected_input[0]) {

       if (v_id.range(1,0) < req0.read().range(1,0))
         v_req[0] = 3; // go to east
       else {
         if (v_id.range(1,0) > req0.read().range(1,0))
           v_req[0] = 5; // go to west
         else {
           if (v_id.range(3,2) < req0.read().range(3,2))
             v_req[0] = 4; // go to south
           else {
             if (v_id.range(3,2) > req0.read().range(3,2))
               v_req[0] = 2; // go to north
             else
               v_req[0] = 1; // that is the destination
           }
         }
  }
}
  // Determine available output based on request
  switch (v_req[0]) {
  case 1:
    v_arbit = v_free & 1;
    break;
  case 2:
    v_arbit = v_free & 2;
    break;
  case 3:
    v_arbit = v_free & 4;
    break;
  case 4:
    v_arbit = v_free & 8;
    break;
  case 5:
    v_arbit = v_free & 16;
    break;
  default:
    break;
  }
  if (!v_connected_input[0]) // if input is not connected
  {
```

```
    if (v_reserved_output[v_req[0]])
      v_arbit = 0; // if the requested output was reserved, go to next input
   }
   if (v_arbit != 0) {
    grant0.write(1); // set grant
    v_select.range(2, 0) = v_req[0];
    v_free = v_free & (~v_arbit); // inactive the related output
    v_connected_input[0] = 1;      // input 0 is connected
    v_reserved_output[v_req[0]] = 1; // output is reserved
    if (req0.read()[5]) {
      printf("Tail flit detected on input 0\n"); // Print statement
      v_connected_input[0] = 0;
      v_reserved_output[v_req[0]] =
         0;
    } // if it is tail flit, reset connection and reservation
   }
}
// Process request from input 1
if (!req1.read()[4]) // if buffer is not empty
{
 printf("Request received on input 1\n"); // Print statement
 if(!v_connected_input[1]){// if input is not connected i.e. it is

     if (v_id.range(1,0) < req1.read().range(1,0))
       v_req[1] = 3; // go to east
     else {
       if (v_id.range(1,0) > req1.read().range(1,0))
         v_req[1] = 5; // go to west
       else {
         if (v_id.range(3,2) < req1.read().range(3,2))
           v_req[1] = 4; // go to south
         else {
           if (v_id.range(3,2) > req1.read().range(3,2))
             v_req[1] = 2; // go to north
           else
             v_req[1] = 1; // that is the destination
         }
       }
     }
}
  // Determine available output based on request
  switch (v_req[1]) {
  case 1:
    v_arbit = v_free & 1;
    break;
  case 2:
    v_arbit = v_free & 2;
    break;
  case 3:
    v_arbit = v_free & 4;
    break;
  case 4:
    v_arbit = v_free & 8;
    break;
  case 5:
    v_arbit = v_free & 16;
    break;
```

```
  default:
    break;
  }
  if (!v_connected_input[1]) // if input is not connected
  {
    if (v_reserved_output[v_req[1]])
      v_arbit = 0; // if the requested output was reserved, go to next input
  }
  if (v_arbit != 0) { // if there is any free output
    grant1.write(1);  // set grant
    v_select.range(5, 3) = v_req[1];
    v_free = v_free & (~v_arbit); // inactive the related outputs
    v_connected_input[1] = 1;      // input 1 is connected
    v_reserved_output[v_req[1]] = 1; // output is reserved
    if (req1.read()[5]) {
      printf("Tail flit detected on input 1\n"); // Print statement
      v_connected_input[1] = 0;
      v_reserved_output[v_req[1]] =
          0;
    } // if it is tail flit, reset connection and reservation
  }
}
// Process request from input 2
if (!req2.read()[4]) // if buffer is not empty
{
  printf("Request received on input 2\n"); // Print statement
  if(!v_connected_input[2]){ // if input is not connected i.e. it is

      if (v_id.range(1,0) < req2.read().range(1,0))
        v_req[2] = 3; // go to east
      else {
        if (v_id.range(1,0) > req2.read().range(1,0))
          v_req[2] = 5; // go to west
        else {
          if (v_id.range(3,2) < req2.read().range(3,2))
            v_req[2] = 4; // go to south
          else {
            if (v_id.range(3,2) > req2.read().range(3,2))
              v_req[2] = 2; // go to north
            else
              v_req[2] = 1; // that is the destination
          }
        }
      }
}
  // Determine available output based on request
  switch (v_req[2]) {
  case 1:
    v_arbit = v_free & 1;
    break;
  case 2:
    v_arbit = v_free & 2;
    break;
  case 3:
    v_arbit = v_free & 4;
    break;
  case 4:
```

```
    v_arbit = v_free & 8;
    break;
  case 5:
    v_arbit = v_free & 16;
    break;
  default:
    break;
  }
  if (!v_connected_input[2]) // if input is not connected
  {
    if (v_reserved_output[v_req[2]])
      v_arbit = 0; // if the requested output was reserved, go to next input
  }
  if (v_arbit != 0) {
    grant2.write(1);  // set grant
    v_select.range(8, 6) = v_req[2];
    v_free = v_free & (~v_arbit); // inactive the related outputs
    v_connected_input[2] = 1;      // input 1 is connected
    v_reserved_output[v_req[2]] = 1; // output is reserved
    if (req2.read()[5]) {
      printf("Tail flit detected on input 2\n"); // Print statement
      v_connected_input[2] = 0;
      v_reserved_output[v_req[2]] =
        0;
    } // if it is tail flit, reset connection and reservation
  }
}
// Process request from input 3
if (!req3.read()[4]) // if buffer is not empty
{
  printf("Request received on input 3\n"); // Print statement
  if(!v_connected_input[3]) {

      if (v_id.range(1,0) < req3.read().range(1,0))
        v_req[3] = 3; // go to east
      else {
        if (v_id.range(1,0) > req3.read().range(1,0))
          v_req[3] = 5; // go to west
        else {
          if (v_id.range(3,2) < req3.read().range(3,2))
            v_req[3] = 4; // go to south
          else {
            if (v_id.range(3,2) > req3.read().range(3,2))
              v_req[3] = 2; // go to north
            else
              v_req[3] = 1; // that is the destination
          }
        }
      }
    }
  // Determine available output based on request
  switch (v_req[3]) {
  case 1:
    v_arbit = v_free & 1;
    break;
  case 2:
    v_arbit = v_free & 2;
```

```
    break;
  case 3:
    v_arbit = v_free & 4;
    break;
  case 4:
    v_arbit = v_free & 8;
    break;
  case 5:
    v_arbit = v_free & 16;
    break;
  default:
    break;
  }
  if (!v_connected_input[3]) // if input is not connected
  {
    if (v_reserved_output[v_req[3]])
      v_arbit = 0; // if the requested output was reserved, go to next input
  }
  if (v_arbit != 0) {
    grant3.write(1);  // set grant
    v_select.range(11, 9) = v_req[3];
    v_free = v_free & (~v_arbit); // inactive the related outputs
    v_connected_input[3] = 1;      // input 3 is connected
    v_reserved_output[v_req[3]] = 1; // output is reserved
    if (req3.read()[5]) {
      printf("Tail flit detected on input 3\n"); // Print statement
      v_connected_input[3] = 0;
      v_reserved_output[v_req[3]] =
        0;
    } // if it is tail flit, reset connection and reservation
  }
}
// Process request from input 4
if (!req4.read()[4]) // if buffer is not empty
{
  printf("Request received on input 4\n"); // Print statement
  if(!v_connected_input[4]){
      if (v_id.range(1,0) < req4.read().range(1,0))
        v_req[4] = 3; // go to east
      else {
        if (v_id.range(1,0) > req4.read().range(1,0))
          v_req[4] = 5; // go to west
        else {
          if (v_id.range(3,2) < req4.read().range(3,2))
            v_req[4] = 4; // go to south
          else {
            if (v_id.range(3,2) > req4.read().range(3,2))
              v_req[4] = 2; // go to north
            else
              v_req[4] = 1; // that is the destination
          }
        }
      }
    }
  // Determine available output based on request
  switch (v_req[4]) {
  case 1:
```

```
      v_arbit = v_free & 1;
      break;
    case 2:
      v_arbit = v_free & 2;
      break;
    case 3:
      v_arbit = v_free & 4;
      break;
    case 4:
      v_arbit = v_free & 8;
      break;
    case 5:
      v_arbit = v_free & 16;
      break;
    default:
      break;
    }
    if (!v_connected_input[4]) // if input is not connected
    {
      if (v_reserved_output[v_req[4]])
        v_arbit = 0; // if the requested output was reserved, go to next input
    }
    if (v_arbit != 0) {
      grant4.write(1);  // set grant
      v_select.range(14, 12) = v_req[4];
      v_free = v_free & (~v_arbit); // inactive the related outputs
                        printf("Selection signal written: %d\n", (int)v_select); // Print statement
      v_connected_input[4] = 1;      // input 4 is connected
      v_reserved_output[v_req[4]] = 1; // output is reserved
      if (req4.read()[5]) {
        printf("Tail flit detected on input 4\n"); // Print statement
        v_connected_input[4] = 0;
        v_reserved_output[v_req[4]] =
            0;
      } // if it is tail flit, reset connection and reservation
    }
   }
   aselect.write(v_select); // Write the selection signal

  }
  printf("Arbiter function ended\n"); // Print statement
}
```

Appendix B

Altered code for 4x4 Torus

Main_noc.cpp

```
// main_noc.cpp (Interactive Version)
```

```cpp
#include "systemc.h"
#include <iostream>
#include <vector>      // For storing exchanges
#include <utility>     // For std::pair
#include <limits>      // For numeric_limits
#include <set>         // For tracking used sources/sinks
#include <string>      // For input line
// Include project headers
#include "packet.h"
#include "source.h"
#include "sink.h"
#include "router.h" // Assuming router.h includes the necessary sub-modules
// Function to safely get integer input from user within a range
int get_validated_int(const std::string& prompt, int min_val, int max_val) {
   int value;
   while (true) {
      std::cout << prompt;
      if (std::cin >> value && value >= min_val && value <= max_val) {
         // Clear the rest of the line, in case user entered extra characters
         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
         return value;
      } else {
         std::cout << "Invalid input. Please enter an integer between "
                   << min_val << " and " << max_val << "." << std::endl;
         std::cin.clear(); // Clear error flags
         // Discard the invalid input
         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
      }
   }
}
int sc_main(int argc, char *argv[])
{
   // --- User Interaction ---
   std::cout << std::endl;
   std::cout << "--------------------------------------------------------------------------------" << std::endl;
   std::cout << " Interactive NoC Simulation Setup (Assumes 4x4 Mesh with 16 Routers)" << std::endl;
   std::cout << "--------------------------------------------------------------------------------" << std::endl;
   int num_exchanges = get_validated_int("Enter the number of source-destination exchanges to configure (1-16): ", 1, 16);
   std::vector<std::pair<int, int>> exchanges;
   std::set<int> used_sources;
   std::set<int> used_sinks;
   for (int i = 0; i < num_exchanges; ++i) {
      std::cout << "\n--- Configuring Exchange #" << (i + 1) << " ---" << std::endl;
      int src_id = -1;
      int dest_id = -1;
      // Get Source ID
      while (true) {
         src_id = get_validated_int("Enter Source ID (0-15): ", 0, 15);
         if (used_sources.find(src_id) == used_sources.end()) {
            break; // Source not used yet
         } else {
            std::cout << "Source " << src_id << " has already been assigned. Please choose another." << std::endl;
         }
      }
      // Get Destination ID
      while (true) {
         dest_id = get_validated_int("Enter Destination Sink ID (0-15): ", 0, 15);
```

```
        if (dest_id == src_id) {
            std::cout << "Destination Sink ID cannot be the same as the Source ID. Please choose another." << std::endl;
        } else if (used_sinks.find(dest_id) == used_sinks.end()) {
            break; // Sink not used yet and different from source
        } else {
            std::cout << "Sink " << dest_id << " has already been assigned. Please choose another." << std::endl;
        }
    }
    // Store valid exchange
    exchanges.push_back({src_id, dest_id});
    used_sources.insert(src_id);
    used_sinks.insert(dest_id);
    std::cout << "Configured: Source " << src_id << " -> Sink " << dest_id << std::endl;
}
// --- Signal Declarations ---
// Clocks
        sc_clock s_clock("S_CLOCK", 125, SC_NS, 0.5, 0.0, SC_NS); // Source clock
        sc_clock r_clock("R_CLOCK", 5, SC_NS, 0.5, 10.0, SC_NS); // Router clock (assuming one clock for all routers)
        sc_clock d_clock("D_CLOCK", 5, SC_NS, 0.5, 10.0, SC_NS); // Destination clock (using same as router for simplicity)
// Signals for Sources (Arrays for 16 sources)
        sc_signal<packet> si_source[16];      // Packet output from sources
        sc_signal<bool>   si_ack_src[16];     // Ack input to sources (from router buffer)
sc_signal<bool>   enable_signals[16];  // Enable signal for sources
sc_signal<sc_uint<4>> dest_signals[16]; // Destination ID signal for sources
sc_signal<sc_uint<4>> source_ids[16];   // Source ID signals
// Signals for Sinks (Arrays for 16 sinks)
        sc_signal<packet> si_sink[16];        // Packet input to sinks
        sc_signal<bool>   si_ack_sink[16];    // Ack output from sinks
sc_signal<sc_uint<4>> sink_ids[16];     // Sink ID signals
// Signals for Router Interconnects (Simplified - focus is on source/sink setup)
        sc_signal<packet> si_input[16]; // Placeholder for router inputs from other routers
        sc_signal<packet> si_output[16]; // Placeholder for router outputs to other routers
sc_signal<bool>   si_ack_in[16]; // Placeholder
sc_signal<bool>   si_ack_ou[16]; // Placeholder
        // --- Module Instantiations ---
// Signals for Inter-Router Connections (4x4 mesh)
// Naming: si_p/a_<FromRouter>_<Direction>_<ToRouter>_<Direction>
// Based on Mapping: 0=L, 1=N, 2=E, 3=S, 4=W

// Horizontal connections (East-West)
// Row 0
sc_signal<packet> si_p_R0E_R1W, si_p_R1W_R0E; sc_signal<bool> si_a_R0E_R1W, si_a_R1W_R0E;
sc_signal<packet> si_p_R1E_R2W, si_p_R2W_R1E; sc_signal<bool> si_a_R1E_R2W, si_a_R2W_R1E;
sc_signal<packet> si_p_R2E_R3W, si_p_R3W_R2E; sc_signal<bool> si_a_R2E_R3W, si_a_R3W_R2E;
// Row 1
sc_signal<packet> si_p_R4E_R5W, si_p_R5W_R4E; sc_signal<bool> si_a_R4E_R5W, si_a_R5W_R4E;
sc_signal<packet> si_p_R5E_R6W, si_p_R6W_R5E; sc_signal<bool> si_a_R5E_R6W, si_a_R6W_R5E;
sc_signal<packet> si_p_R6E_R7W, si_p_R7W_R6E; sc_signal<bool> si_a_R6E_R7W, si_a_R7W_R6E;
// Row 2
sc_signal<packet> si_p_R8E_R9W, si_p_R9W_R8E; sc_signal<bool> si_a_R8E_R9W, si_a_R9W_R8E;
sc_signal<packet> si_p_R9E_R10W, si_p_R10W_R9E; sc_signal<bool> si_a_R9E_R10W, si_a_R10W_R9E;
sc_signal<packet> si_p_R10E_R11W, si_p_R11W_R10E; sc_signal<bool> si_a_R10E_R11W, si_a_R11W_R10E;
// Row 3
sc_signal<packet> si_p_R12E_R13W, si_p_R13W_R12E; sc_signal<bool> si_a_R12E_R13W, si_a_R13W_R12E;
sc_signal<packet> si_p_R13E_R14W, si_p_R14W_R13E; sc_signal<bool> si_a_R13E_R14W, si_a_R14W_R13E;
sc_signal<packet> si_p_R14E_R15W, si_p_R15W_R14E; sc_signal<bool> si_a_R14E_R15W, si_a_R15W_R14E;
```

```
// Vertical connections (North-South)
// Column 0
sc_signal<packet> si_p_R0S_R4N, si_p_R4N_R0S; sc_signal<bool> si_a_R0S_R4N, si_a_R4N_R0S;
sc_signal<packet> si_p_R4S_R8N, si_p_R8N_R4S; sc_signal<bool> si_a_R4S_R8N, si_a_R8N_R4S;
sc_signal<packet> si_p_R8S_R12N, si_p_R12N_R8S; sc_signal<bool> si_a_R8S_R12N, si_a_R12N_R8S;

// Torus wrap-around connections (North-South)
// Top row to bottom row
sc_signal<packet> si_p_R0N_R12S, si_p_R12S_R0N; sc_signal<bool> si_a_R0N_R12S, si_a_R12S_R0N;
sc_signal<packet> si_p_R1N_R13S, si_p_R13S_R1N; sc_signal<bool> si_a_R1N_R13S, si_a_R13S_R1N;
sc_signal<packet> si_p_R2N_R14S, si_p_R14S_R2N; sc_signal<bool> si_a_R2N_R14S, si_a_R14S_R2N;
sc_signal<packet> si_p_R3N_R15S, si_p_R15S_R3N; sc_signal<bool> si_a_R3N_R15S, si_a_R15S_R3N;

// Torus wrap-around connections (East-West)
// Left column to right column
sc_signal<packet> si_p_R0W_R3E, si_p_R3E_R0W; sc_signal<bool> si_a_R0W_R3E, si_a_R3E_R0W;
sc_signal<packet> si_p_R4W_R7E, si_p_R7E_R4W; sc_signal<bool> si_a_R4W_R7E, si_a_R7E_R4W;
sc_signal<packet> si_p_R8W_R11E, si_p_R11E_R8W; sc_signal<bool> si_a_R8W_R11E, si_a_R11E_R8W;
sc_signal<packet> si_p_R12W_R15E, si_p_R15E_R12W; sc_signal<bool> si_a_R12W_R15E, si_a_R15E_R12W;
// Column 1
sc_signal<packet> si_p_R1S_R5N, si_p_R5N_R1S; sc_signal<bool> si_a_R1S_R5N, si_a_R5N_R1S;
sc_signal<packet> si_p_R5S_R9N, si_p_R9N_R5S; sc_signal<bool> si_a_R5S_R9N, si_a_R9N_R5S;
sc_signal<packet> si_p_R9S_R13N, si_p_R13N_R9S; sc_signal<bool> si_a_R9S_R13N, si_a_R13N_R9S;
// Column 2
sc_signal<packet> si_p_R2S_R6N, si_p_R6N_R2S; sc_signal<bool> si_a_R2S_R6N, si_a_R6N_R2S;
sc_signal<packet> si_p_R6S_R10N, si_p_R10N_R6S; sc_signal<bool> si_a_R6S_R10N, si_a_R10N_R6S;
sc_signal<packet> si_p_R10S_R14N, si_p_R14N_R10S; sc_signal<bool> si_a_R10S_R14N, si_a_R14N_R10S;
// Column 3
sc_signal<packet> si_p_R3S_R7N, si_p_R7N_R3S; sc_signal<bool> si_a_R3S_R7N, si_a_R7N_R3S;
sc_signal<packet> si_p_R7S_R11N, si_p_R11N_R7S; sc_signal<bool> si_a_R7S_R11N, si_a_R11N_R7S;
sc_signal<packet> si_p_R11S_R15N, si_p_R15N_R11S; sc_signal<bool> si_a_R11S_R15N, si_a_R15N_R11S;


// Instantiate Sources (0 to 3 only, as per original implementation)
source* sources[16];
for (int i = 0; i < 16; ++i) {
    std::string name = "source" + std::to_string(i);
    sources[i] = new source(name.c_str());
    sources[i]->packet_out(si_source[i]);
    sources[i]->source_id(source_ids[i]);
    sources[i]->ach_in(si_ack_src[i]);
    sources[i]->CLK(s_clock);
    sources[i]->enable_source(enable_signals[i]);     // Connect enable port
    sources[i]->target_dest_id(dest_signals[i]); // Connect destination port
}
// Instantiate Sinks (0 to 15 only, as per original implementation)
sink* sinks[16];
for (int i = 0; i < 16; ++i) {
    std::string name = "sink" + std::to_string(i);
    sinks[i] = new sink(name.c_str());
    sinks[i]->packet_in(si_sink[i]);
    sinks[i]->ack_out(si_ack_sink[i]);
    sinks[i]->sink_id(sink_ids[i]);
    sinks[i]->sclk(d_clock); // Using destination clock
}
// Instantiate Routers (0 to 15) - 4x4 Mesh
router* routers[16];
```

```
for (int i = 0; i < 16; ++i) {
    std::string name = "router" + std::to_string(i);
    routers[i] = new router(name.c_str());
    // Connect Clock for all routers
    routers[i]->rclk(r_clock);

    // Connect Router ID - For now, using modulo 4 to match with available sources/sinks
    routers[i]->router_id(source_ids[i % 16]);
}

// Connect sources and sinks only to the first 4 routers (as per original implementation)
for (int i = 0; i < 16; ++i) {
    // Connect local source/sink
    routers[i]->in0(si_source[i]);      // Input from local source
    routers[i]->outack0(si_ack_src[i]); // Ack back to local source
    routers[i]->out0(si_sink[i]);       // Output to local sink
    routers[i]->inack0(si_ack_sink[i]); // Ack from local sink
}
// --- Configuration before Simulation ---
// Assign IDs
for (int i = 0; i < 16; ++i) {
    source_ids[i].write(i);
    sink_ids[i].write(i);
    // Initialize control signals
    enable_signals[i].write(false); // Disable all sources initially
    dest_signals[i].write(0);       // Default destination
}
// Enable and configure sources based on user input
for (const auto& exchange : exchanges) {
    int src = exchange.first;
    int dest = exchange.second;
    enable_signals[src].write(true);
    dest_signals[src].write(dest);
}
// Connect all routers in the 4x4 mesh
// Horizontal connections (East-West)
// Row 0
routers[0]->out2(si_p_R0E_R1W);    routers[0]->inack2(si_a_R1W_R0E);
routers[1]->in4(si_p_R0E_R1W);     routers[1]->outack4(si_a_R1W_R0E);
routers[1]->out4(si_p_R1W_R0E);    routers[1]->inack4(si_a_R0E_R1W);
routers[0]->in2(si_p_R1W_R0E);     routers[0]->outack2(si_a_R0E_R1W);

routers[1]->out2(si_p_R1E_R2W);    routers[1]->inack2(si_a_R2W_R1E);
routers[2]->in4(si_p_R1E_R2W);     routers[2]->outack4(si_a_R2W_R1E);
routers[2]->out4(si_p_R2W_R1E);    routers[2]->inack4(si_a_R1E_R2W);
routers[1]->in2(si_p_R2W_R1E);     routers[1]->outack2(si_a_R1E_R2W);

routers[2]->out2(si_p_R2E_R3W);    routers[2]->inack2(si_a_R3W_R2E);
routers[3]->in4(si_p_R2E_R3W);     routers[3]->outack4(si_a_R3W_R2E);
routers[3]->out4(si_p_R3W_R2E);    routers[3]->inack4(si_a_R2E_R3W);
routers[2]->in2(si_p_R3W_R2E);     routers[2]->outack2(si_a_R2E_R3W);

// Row 1
routers[4]->out2(si_p_R4E_R5W);    routers[4]->inack2(si_a_R5W_R4E);
routers[5]->in4(si_p_R4E_R5W);     routers[5]->outack4(si_a_R5W_R4E);
routers[5]->out4(si_p_R5W_R4E);    routers[5]->inack4(si_a_R4E_R5W);
routers[4]->in2(si_p_R5W_R4E);     routers[4]->outack2(si_a_R4E_R5W);
```

```
routers[5]->out2(si_p_R5E_R6W);    routers[5]->inack2(si_a_R6W_R5E);
routers[6]->in4(si_p_R5E_R6W);     routers[6]->outack4(si_a_R6W_R5E);
routers[6]->out4(si_p_R6W_R5E);    routers[6]->inack4(si_a_R5E_R6W);
routers[5]->in2(si_p_R6W_R5E);     routers[5]->outack2(si_a_R5E_R6W);

routers[6]->out2(si_p_R6E_R7W);    routers[6]->inack2(si_a_R7W_R6E);
routers[7]->in4(si_p_R6E_R7W);     routers[7]->outack4(si_a_R7W_R6E);
routers[7]->out4(si_p_R7W_R6E);    routers[7]->inack4(si_a_R6E_R7W);
routers[6]->in2(si_p_R7W_R6E);     routers[6]->outack2(si_a_R6E_R7W);

// Row 2
routers[8]->out2(si_p_R8E_R9W);    routers[8]->inack2(si_a_R9W_R8E);
routers[9]->in4(si_p_R8E_R9W);     routers[9]->outack4(si_a_R9W_R8E);
routers[9]->out4(si_p_R9W_R8E);    routers[9]->inack4(si_a_R8E_R9W);
routers[8]->in2(si_p_R9W_R8E);     routers[8]->outack2(si_a_R8E_R9W);

routers[9]->out2(si_p_R9E_R10W);   routers[9]->inack2(si_a_R10W_R9E);
routers[10]->in4(si_p_R9E_R10W);   routers[10]->outack4(si_a_R10W_R9E);
routers[10]->out4(si_p_R10W_R9E);  routers[10]->inack4(si_a_R9E_R10W);
routers[9]->in2(si_p_R10W_R9E);    routers[9]->outack2(si_a_R9E_R10W);

routers[10]->out2(si_p_R10E_R11W); routers[10]->inack2(si_a_R11W_R10E);
routers[11]->in4(si_p_R10E_R11W);  routers[11]->outack4(si_a_R11W_R10E);
routers[11]->out4(si_p_R11W_R10E); routers[11]->inack4(si_a_R10E_R11W);
routers[10]->in2(si_p_R11W_R10E);  routers[10]->outack2(si_a_R10E_R11W);

// Row 3
routers[12]->out2(si_p_R12E_R13W); routers[12]->inack2(si_a_R13W_R12E);
routers[13]->in4(si_p_R12E_R13W);  routers[13]->outack4(si_a_R13W_R12E);
routers[13]->out4(si_p_R13W_R12E); routers[13]->inack4(si_a_R12E_R13W);
routers[12]->in2(si_p_R13W_R12E);  routers[12]->outack2(si_a_R12E_R13W);

routers[13]->out2(si_p_R13E_R14W); routers[13]->inack2(si_a_R14W_R13E);
routers[14]->in4(si_p_R13E_R14W);  routers[14]->outack4(si_a_R14W_R13E);
routers[14]->out4(si_p_R14W_R13E); routers[14]->inack4(si_a_R13E_R14W);
routers[13]->in2(si_p_R14W_R13E);  routers[13]->outack2(si_a_R13E_R14W);

routers[14]->out2(si_p_R14E_R15W); routers[14]->inack2(si_a_R15W_R14E);
routers[15]->in4(si_p_R14E_R15W);  routers[15]->outack4(si_a_R15W_R14E);
routers[15]->out4(si_p_R15W_R14E); routers[15]->inack4(si_a_R14E_R15W);
routers[14]->in2(si_p_R15W_R14E);  routers[14]->outack2(si_a_R14E_R15W);
// Vertical connections (North-South)
// Column 0
routers[0]->out3(si_p_R0S_R4N);    routers[0]->inack3(si_a_R4N_R0S);
routers[4]->in1(si_p_R0S_R4N);     routers[4]->outack1(si_a_R4N_R0S);
routers[4]->out1(si_p_R4N_R0S);    routers[4]->inack1(si_a_R0S_R4N);
routers[0]->in3(si_p_R4N_R0S);     routers[0]->outack3(si_a_R0S_R4N);

routers[4]->out3(si_p_R4S_R8N);    routers[4]->inack3(si_a_R8N_R4S);
routers[8]->in1(si_p_R4S_R8N);     routers[8]->outack1(si_a_R8N_R4S);
routers[8]->out1(si_p_R8N_R4S);    routers[8]->inack1(si_a_R4S_R8N);
routers[4]->in3(si_p_R8N_R4S);     routers[4]->outack3(si_a_R4S_R8N);

routers[8]->out3(si_p_R8S_R12N);   routers[8]->inack3(si_a_R12N_R8S);
routers[12]->in1(si_p_R8S_R12N);   routers[12]->outack1(si_a_R12N_R8S);
routers[12]->out1(si_p_R12N_R8S);  routers[12]->inack1(si_a_R8S_R12N);
```

```
routers[8]->in3(si_p_R12N_R8S);     routers[8]->outack3(si_a_R8S_R12N);

// Column 1
routers[1]->out3(si_p_R1S_R5N);     routers[1]->inack3(si_a_R5N_R1S);
routers[5]->in1(si_p_R1S_R5N);      routers[5]->outack1(si_a_R5N_R1S);
routers[5]->out1(si_p_R5N_R1S);     routers[5]->inack1(si_a_R1S_R5N);
routers[1]->in3(si_p_R5N_R1S);      routers[1]->outack3(si_a_R1S_R5N);

routers[5]->out3(si_p_R5S_R9N);     routers[5]->inack3(si_a_R9N_R5S);
routers[9]->in1(si_p_R5S_R9N);      routers[9]->outack1(si_a_R9N_R5S);
routers[9]->out1(si_p_R9N_R5S);     routers[9]->inack1(si_a_R5S_R9N);
routers[5]->in3(si_p_R9N_R5S);      routers[5]->outack3(si_a_R5S_R9N);

routers[9]->out3(si_p_R9S_R13N);    routers[9]->inack3(si_a_R13N_R9S);
routers[13]->in1(si_p_R9S_R13N);    routers[13]->outack1(si_a_R13N_R9S);
routers[13]->out1(si_p_R13N_R9S);   routers[13]->inack1(si_a_R9S_R13N);
routers[9]->in3(si_p_R13N_R9S);     routers[9]->outack3(si_a_R9S_R13N);

// Column 2
routers[2]->out3(si_p_R2S_R6N);     routers[2]->inack3(si_a_R6N_R2S);
routers[6]->in1(si_p_R2S_R6N);      routers[6]->outack1(si_a_R6N_R2S);
routers[6]->out1(si_p_R6N_R2S);     routers[6]->inack1(si_a_R2S_R6N);
routers[2]->in3(si_p_R6N_R2S);      routers[2]->outack3(si_a_R2S_R6N);

routers[6]->out3(si_p_R6S_R10N);    routers[6]->inack3(si_a_R10N_R6S);
routers[10]->in1(si_p_R6S_R10N);    routers[10]->outack1(si_a_R10N_R6S);
routers[10]->out1(si_p_R10N_R6S);   routers[10]->inack1(si_a_R6S_R10N);
routers[6]->in3(si_p_R10N_R6S);     routers[6]->outack3(si_a_R6S_R10N);

routers[10]->out3(si_p_R10S_R14N);  routers[10]->inack3(si_a_R14N_R10S);
routers[14]->in1(si_p_R10S_R14N);   routers[14]->outack1(si_a_R14N_R10S);
routers[14]->out1(si_p_R14N_R10S);  routers[14]->inack1(si_a_R10S_R14N);
routers[10]->in3(si_p_R14N_R10S);   routers[10]->outack3(si_a_R10S_R14N);

// Column 3
routers[3]->out3(si_p_R3S_R7N);     routers[3]->inack3(si_a_R7N_R3S);
routers[7]->in1(si_p_R3S_R7N);      routers[7]->outack1(si_a_R7N_R3S);
routers[7]->out1(si_p_R7N_R3S);     routers[7]->inack1(si_a_R3S_R7N);
routers[3]->in3(si_p_R7N_R3S);      routers[3]->outack3(si_a_R3S_R7N);

routers[7]->out3(si_p_R7S_R11N);    routers[7]->inack3(si_a_R11N_R7S);
routers[11]->in1(si_p_R7S_R11N);    routers[11]->outack1(si_a_R11N_R7S);
routers[11]->out1(si_p_R11N_R7S);   routers[11]->inack1(si_a_R7S_R11N);
routers[7]->in3(si_p_R11N_R7S);     routers[7]->outack3(si_a_R7S_R11N);

routers[11]->out3(si_p_R11S_R15N);  routers[11]->inack3(si_a_R15N_R11S);
routers[15]->in1(si_p_R11S_R15N);   routers[15]->outack1(si_a_R15N_R11S);
routers[15]->out1(si_p_R15N_R11S);  routers[15]->inack1(si_a_R11S_R15N);
routers[11]->in3(si_p_R15N_R11S);   routers[11]->outack3(si_a_R11S_R15N);
printf("Inter-Router Ports Connected\n");

// Connect Edge Ports to Dummies
// Top row routers (North edge)
// Torus connections for top and bottom rows (North-South wrap)
// Connect Router 0 North (port 1) to Router 12 South (port 3)
routers[0]->out1(si_p_R0N_R12S);    routers[0]->inack1(si_a_R12S_R0N);
routers[12]->in3(si_p_R0N_R12S);    routers[12]->outack3(si_a_R12S_R0N);
```

```
routers[12]->out3(si_p_R12S_R0N);  routers[12]->inack3(si_a_R0N_R12S);
routers[0]->in1(si_p_R12S_R0N);    routers[0]->outack1(si_a_R0N_R12S);
// Connect Router 1 North (port 1) to Router 13 South (port 3)
routers[1]->out1(si_p_R1N_R13S);   routers[1]->inack1(si_a_R13S_R1N);
routers[13]->in3(si_p_R1N_R13S);   routers[13]->outack3(si_a_R13S_R1N);
routers[13]->out3(si_p_R13S_R1N);  routers[13]->inack3(si_a_R1N_R13S);
routers[1]->in1(si_p_R13S_R1N);    routers[1]->outack1(si_a_R1N_R13S);
// Connect Router 2 North (port 1) to Router 14 South (port 3)
routers[2]->out1(si_p_R2N_R14S);   routers[2]->inack1(si_a_R14S_R2N);
routers[14]->in3(si_p_R2N_R14S);   routers[14]->outack3(si_a_R14S_R2N);
routers[14]->out3(si_p_R14S_R2N);  routers[14]->inack3(si_a_R2N_R14S);
routers[2]->in1(si_p_R14S_R2N);    routers[2]->outack1(si_a_R2N_R14S);
// Connect Router 3 North (port 1) to Router 15 South (port 3)
routers[3]->out1(si_p_R3N_R15S);   routers[3]->inack1(si_a_R15S_R3N);
routers[15]->in3(si_p_R3N_R15S);   routers[15]->outack3(si_a_R15S_R3N);
routers[15]->out3(si_p_R15S_R3N);  routers[15]->inack3(si_a_R3N_R15S);
routers[3]->in1(si_p_R15S_R3N);    routers[3]->outack1(si_a_R3N_R15S);
// Torus connections for left and right columns (East-West wrap)
// Connect Router 0 West (port 4) to Router 3 East (port 2)
routers[0]->out4(si_p_R0W_R3E);    routers[0]->inack4(si_a_R3E_R0W);
routers[3]->in2(si_p_R0W_R3E);     routers[3]->outack2(si_a_R3E_R0W);
routers[3]->out2(si_p_R3E_R0W);    routers[3]->inack2(si_a_R0W_R3E);
routers[0]->in4(si_p_R3E_R0W);     routers[0]->outack4(si_a_R0W_R3E);
// Connect Router 4 West (port 4) to Router 7 East (port 2)
routers[4]->out4(si_p_R4W_R7E);    routers[4]->inack4(si_a_R7E_R4W);
routers[7]->in2(si_p_R4W_R7E);     routers[7]->outack2(si_a_R7E_R4W);
routers[7]->out2(si_p_R7E_R4W);    routers[7]->inack2(si_a_R4W_R7E);
routers[4]->in4(si_p_R7E_R4W);     routers[4]->outack4(si_a_R4W_R7E);
// Connect Router 8 West (port 4) to Router 11 East (port 2)
routers[8]->out4(si_p_R8W_R11E);   routers[8]->inack4(si_a_R11E_R8W);
routers[11]->in2(si_p_R8W_R11E);   routers[11]->outack2(si_a_R11E_R8W);
routers[11]->out2(si_p_R11E_R8W);  routers[11]->inack2(si_a_R8W_R11E);
routers[8]->in4(si_p_R11E_R8W);    routers[8]->outack4(si_a_R8W_R11E);
// Connect Router 12 West (port 4) to Router 15 East (port 2)
routers[12]->out4(si_p_R12W_R15E); routers[12]->inack4(si_a_R15E_R12W);
routers[15]->in2(si_p_R12W_R15E);  routers[15]->outack2(si_a_R15E_R12W);
routers[15]->out2(si_p_R15E_R12W); routers[15]->inack2(si_a_R12W_R15E);
routers[12]->in4(si_p_R15E_R12W);  routers[12]->outack4(si_a_R12W_R15E);
printf("Torus Wrap-Around Ports Connected\n");
printf("All Module ports connected in Torus Topology\n");
// --- Tracing (Optional) ---
        sc_trace_file *tf = sc_create_vcd_trace_file("interactive_noc_trace");
        if (tf) {
  sc_trace(tf, s_clock, "s_clock");
  sc_trace(tf, r_clock, "r_clock");
  sc_trace(tf, d_clock, "d_clock");
  for (int i = 0; i < 16; ++i) {
    sc_trace(tf, si_source[i], "source" + std::to_string(i) + "_packet_out");
    sc_trace(tf, si_sink[i], "sink" + std::to_string(i) + "_packet_in");
    sc_trace(tf, si_ack_src[i], "source" + std::to_string(i) + "_ack_in");
    sc_trace(tf, si_ack_sink[i], "sink" + std::to_string(i) + "_ack_out");
    sc_trace(tf, enable_signals[i], "source" + std::to_string(i) + "_enable");
    sc_trace(tf, dest_signals[i], "source" + std::to_string(i) + "_dest_id");
    sc_trace(tf, source_ids[i], "source" + std::to_string(i) + "_id"); // Added source_ids trace
    sc_trace(tf, sink_ids[i], "sink" + std::to_string(i) + "_id");     // Added sink_ids trace
    // Trace placeholder router signals
    sc_trace(tf, si_input[i], "router" + std::to_string(i) + "_input_placeholder");
```

```
            sc_trace(tf, si_output[i], "router" + std::to_string(i) + "_output_placeholder");
            sc_trace(tf, si_ack_in[i], "router" + std::to_string(i) + "_ack_in_placeholder");
            sc_trace(tf, si_ack_ou[i], "router" + std::to_string(i) + "_ack_out_placeholder");
        }
        // Trace Inter-Router Signals
        // Horizontal Row 0
        sc_trace(tf, si_p_R0E_R1W, "si_p_R0E_R1W"); sc_trace(tf, si_p_R1W_R0E, "si_p_R1W_R0E"); sc_trace(tf, si_a_R0E_R1W, "si_a_R0E_R1W"); sc_trace(tf, si_a_R1W_R0E, "si_a_R1W_R0E");
        sc_trace(tf, si_p_R1E_R2W, "si_p_R1E_R2W"); sc_trace(tf, si_p_R2W_R1E, "si_p_R2W_R1E"); sc_trace(tf, si_a_R1E_R2W, "si_a_R1E_R2W"); sc_trace(tf, si_a_R2W_R1E, "si_a_R2W_R1E");
        sc_trace(tf, si_p_R2E_R3W, "si_p_R2E_R3W"); sc_trace(tf, si_p_R3W_R2E, "si_p_R3W_R2E"); sc_trace(tf, si_a_R2E_R3W, "si_a_R2E_R3W"); sc_trace(tf, si_a_R3W_R2E, "si_a_R3W_R2E");
        // Horizontal Row 1
        sc_trace(tf, si_p_R4E_R5W, "si_p_R4E_R5W"); sc_trace(tf, si_p_R5W_R4E, "si_p_R5W_R4E"); sc_trace(tf, si_a_R4E_R5W, "si_a_R4E_R5W"); sc_trace(tf, si_a_R5W_R4E, "si_a_R5W_R4E");
        sc_trace(tf, si_p_R5E_R6W, "si_p_R5E_R6W"); sc_trace(tf, si_p_R6W_R5E, "si_p_R6W_R5E"); sc_trace(tf, si_a_R5E_R6W, "si_a_R5E_R6W"); sc_trace(tf, si_a_R6W_R5E, "si_a_R6W_R5E");
        sc_trace(tf, si_p_R6E_R7W, "si_p_R6E_R7W"); sc_trace(tf, si_p_R7W_R6E, "si_p_R7W_R6E"); sc_trace(tf, si_a_R6E_R7W, "si_a_R6E_R7W"); sc_trace(tf, si_a_R7W_R6E, "si_a_R7W_R6E");
        // Horizontal Row 2
        sc_trace(tf, si_p_R8E_R9W, "si_p_R8E_R9W"); sc_trace(tf, si_p_R9W_R8E, "si_p_R9W_R8E"); sc_trace(tf, si_a_R8E_R9W, "si_a_R8E_R9W"); sc_trace(tf, si_a_R9W_R8E, "si_a_R9W_R8E");
        sc_trace(tf, si_p_R9E_R10W, "si_p_R9E_R10W"); sc_trace(tf, si_p_R10W_R9E, "si_p_R10W_R9E"); sc_trace(tf, si_a_R9E_R10W, "si_a_R9E_R10W"); sc_trace(tf, si_a_R10W_R9E, "si_a_R10W_R9E");
        sc_trace(tf, si_p_R10E_R11W, "si_p_R10E_R11W"); sc_trace(tf, si_p_R11W_R10E, "si_p_R11W_R10E"); sc_trace(tf, si_a_R10E_R11W, "si_a_R10E_R11W"); sc_trace(tf, si_a_R11W_R10E, "si_a_R11W_R10E");
        // Horizontal Row 3
        sc_trace(tf, si_p_R12E_R13W, "si_p_R12E_R13W"); sc_trace(tf, si_p_R13W_R12E, "si_p_R13W_R12E"); sc_trace(tf, si_a_R12E_R13W, "si_a_R12E_R13W"); sc_trace(tf, si_a_R13W_R12E, "si_a_R13W_R12E");
        sc_trace(tf, si_p_R13E_R14W, "si_p_R13E_R14W"); sc_trace(tf, si_p_R14W_R13E, "si_p_R14W_R13E"); sc_trace(tf, si_a_R13E_R14W, "si_a_R13E_R14W"); sc_trace(tf, si_a_R14W_R13E, "si_a_R14W_R13E");
        sc_trace(tf, si_p_R14E_R15W, "si_p_R14E_R15W"); sc_trace(tf, si_p_R15W_R14E, "si_p_R15W_R14E"); sc_trace(tf, si_a_R14E_R15W, "si_a_R14E_R15W"); sc_trace(tf, si_a_R15W_R14E, "si_a_R15W_R14E");
        // Vertical Column 0
        sc_trace(tf, si_p_R0S_R4N, "si_p_R0S_R4N"); sc_trace(tf, si_p_R4N_R0S, "si_p_R4N_R0S"); sc_trace(tf, si_a_R0S_R4N, "si_a_R0S_R4N"); sc_trace(tf, si_a_R4N_R0S, "si_a_R4N_R0S");
        sc_trace(tf, si_p_R4S_R8N, "si_p_R4S_R8N"); sc_trace(tf, si_p_R8N_R4S, "si_p_R8N_R4S"); sc_trace(tf, si_a_R4S_R8N, "si_a_R4S_R8N"); sc_trace(tf, si_a_R8N_R4S, "si_a_R8N_R4S");
        sc_trace(tf, si_p_R8S_R12N, "si_p_R8S_R12N"); sc_trace(tf, si_p_R12N_R8S, "si_p_R12N_R8S"); sc_trace(tf, si_a_R8S_R12N, "si_a_R8S_R12N"); sc_trace(tf, si_a_R12N_R8S, "si_a_R12N_R8S");
        // Vertical Column 1
        sc_trace(tf, si_p_R1S_R5N, "si_p_R1S_R5N"); sc_trace(tf, si_p_R5N_R1S, "si_p_R5N_R1S"); sc_trace(tf, si_a_R1S_R5N, "si_a_R1S_R5N"); sc_trace(tf, si_a_R5N_R1S, "si_a_R5N_R1S");
        sc_trace(tf, si_p_R5S_R9N, "si_p_R5S_R9N"); sc_trace(tf, si_p_R9N_R5S, "si_p_R9N_R5S"); sc_trace(tf, si_a_R5S_R9N, "si_a_R5S_R9N"); sc_trace(tf, si_a_R9N_R5S, "si_a_R9N_R5S");
        sc_trace(tf, si_p_R9S_R13N, "si_p_R9S_R13N"); sc_trace(tf, si_p_R13N_R9S, "si_p_R13N_R9S"); sc_trace(tf, si_a_R9S_R13N, "si_a_R9S_R13N"); sc_trace(tf, si_a_R13N_R9S, "si_a_R13N_R9S");
        // Vertical Column 2
        sc_trace(tf, si_p_R2S_R6N, "si_p_R2S_R6N"); sc_trace(tf, si_p_R6N_R2S, "si_p_R6N_R2S"); sc_trace(tf, si_a_R2S_R6N, "si_a_R2S_R6N"); sc_trace(tf, si_a_R6N_R2S, "si_a_R6N_R2S");
        sc_trace(tf, si_p_R6S_R10N, "si_p_R6S_R10N"); sc_trace(tf, si_p_R10N_R6S, "si_p_R10N_R6S"); sc_trace(tf, si_a_R6S_R10N, "si_a_R6S_R10N"); sc_trace(tf, si_a_R10N_R6S, "si_a_R10N_R6S");
        sc_trace(tf, si_p_R10S_R14N, "si_p_R10S_R14N"); sc_trace(tf, si_p_R14N_R10S, "si_p_R14N_R10S"); sc_trace(tf, si_a_R10S_R14N, "si_a_R10S_R14N"); sc_trace(tf, si_a_R14N_R10S, "si_a_R14N_R10S");
        // Vertical Column 3
        sc_trace(tf, si_p_R3S_R7N, "si_p_R3S_R7N"); sc_trace(tf, si_p_R7N_R3S, "si_p_R7N_R3S"); sc_trace(tf, si_a_R3S_R7N, "si_a_R3S_R7N"); sc_trace(tf, si_a_R7N_R3S, "si_a_R7N_R3S");
```

```
        sc_trace(tf, si_p_R7S_R11N, "si_p_R7S_R11N"); sc_trace(tf, si_p_R11N_R7S, "si_p_R11N_R7S"); sc_trace(tf, si_a_R7S_R11N, "si_a_R7S_R11N"); sc_trace(tf, si_a_R11N_R7S, "si_a_R11N_R7S");
        sc_trace(tf, si_p_R11S_R15N, "si_p_R11S_R15N"); sc_trace(tf, si_p_R15N_R11S, "si_p_R15N_R11S"); sc_trace(tf, si_a_R11S_R15N, "si_a_R11S_R15N"); sc_trace(tf, si_a_R15N_R11S, "si_a_R15N_R11S");
        // Torus N-S
        sc_trace(tf, si_p_R0N_R12S, "si_p_R0N_R12S"); sc_trace(tf, si_p_R12S_R0N, "si_p_R12S_R0N"); sc_trace(tf, si_a_R0N_R12S, "si_a_R0N_R12S"); sc_trace(tf, si_a_R12S_R0N, "si_a_R12S_R0N");
        sc_trace(tf, si_p_R1N_R13S, "si_p_R1N_R13S"); sc_trace(tf, si_p_R13S_R1N, "si_p_R13S_R1N"); sc_trace(tf, si_a_R1N_R13S, "si_a_R1N_R13S"); sc_trace(tf, si_a_R13S_R1N, "si_a_R13S_R1N");
        sc_trace(tf, si_p_R2N_R14S, "si_p_R2N_R14S"); sc_trace(tf, si_p_R14S_R2N, "si_p_R14S_R2N"); sc_trace(tf, si_a_R2N_R14S, "si_a_R2N_R14S"); sc_trace(tf, si_a_R14S_R2N, "si_a_R14S_R2N");
        sc_trace(tf, si_p_R3N_R15S, "si_p_R3N_R15S"); sc_trace(tf, si_p_R15S_R3N, "si_p_R15S_R3N"); sc_trace(tf, si_a_R3N_R15S, "si_a_R3N_R15S"); sc_trace(tf, si_a_R15S_R3N, "si_a_R15S_R3N");
        // Torus E-W
        sc_trace(tf, si_p_R0W_R3E, "si_p_R0W_R3E"); sc_trace(tf, si_p_R3E_R0W, "si_p_R3E_R0W"); sc_trace(tf, si_a_R0W_R3E, "si_a_R0W_R3E"); sc_trace(tf, si_a_R3E_R0W, "si_a_R3E_R0W");
        sc_trace(tf, si_p_R4W_R7E, "si_p_R4W_R7E"); sc_trace(tf, si_p_R7E_R4W, "si_p_R7E_R4W"); sc_trace(tf, si_a_R4W_R7E, "si_a_R4W_R7E"); sc_trace(tf, si_a_R7E_R4W, "si_a_R7E_R4W");
        sc_trace(tf, si_p_R8W_R11E, "si_p_R8W_R11E"); sc_trace(tf, si_p_R11E_R8W, "si_p_R11E_R8W"); sc_trace(tf, si_a_R8W_R11E, "si_a_R8W_R11E"); sc_trace(tf, si_a_R11E_R8W, "si_a_R11E_R8W");
        sc_trace(tf, si_p_R12W_R15E, "si_p_R12W_R15E"); sc_trace(tf, si_p_R15E_R12W, "si_p_R15E_R12W"); sc_trace(tf, si_a_R12W_R15E, "si_a_R12W_R15E"); sc_trace(tf, si_a_R15E_R12W, "si_a_R15E_R12W");

    } else {
        std::cerr << "Warning: Could not create VCD trace file." << std::endl;
    }
    // --- Simulation Start ---
            std::cout << "\nStarting simulation..." << std::endl;
            std::cout << "Press \"Return\" key to start the simulation..." << std::endl;
    // Clear the input buffer before getchar
    std::cin.sync(); // Or other appropriate method depending on OS/compiler
            getchar();
            sc_start(10*125 + 124, SC_NS);  // Run for a specific duration
    if (tf) {
                sc_close_vcd_trace_file(tf);
    }
    // --- Simulation End & Results ---
            std::cout << std::endl << std::endl << "--------------------------------------------------------------------------------" << std::endl;
            std::cout << "End of Simulation..." << std::endl;
    // Display configured exchanges
    std::cout << "\nConfigured Exchanges:" << std::endl;
    if (exchanges.empty()) {
        std::cout << "  None" << std::endl;
    } else {
        for (const auto& exchange : exchanges) {
            std::cout << "  Source " << exchange.first << " -> Sink " << exchange.second << std::endl;
        }
    }
    // Display packet counts (summing from active sources/sinks)
    long long total_sent = 0;
    long long total_recv = 0;
    for (int i = 0; i < 16; ++i) {
        if (enable_signals[i].read()) { // Check if the source was enabled
            total_sent += sources[i]->pkt_snt;
        }
        // Check if the sink was a target in any exchange
        bool sink_was_target = false;
```

```
        for (const auto& exchange : exchanges) {
            if (exchange.second == i) {
                sink_was_target = true;
                break;
            }
        }
        if (sink_was_target) {
            total_recv += sinks[i]->pkt_recv;
        }
    }
                std::cout << "\nTotal number of packets sent by enabled sources: " << total_sent << std::endl;
                std::cout << "Total number of packets received by targeted sinks: " << total_recv << std::endl;
                std::cout << "-------------------------------------------------------------------------------" << std::endl;
    std::cout << "  Press \"Return\" key to end the program..." << std::endl << std::endl;
    // Clear the input buffer before getchar
    std::cin.sync();
                getchar();
    // Clean up dynamically allocated modules
    for (int i = 0; i < 16; ++i) {
        delete sources[i];
        delete sinks[i];
    }

    // Clean up all routers (0-15)
    for (int i = 0; i < 16; ++i) {
        delete routers[i];
    }
    return 0;
}
```

Arbiter.cpp

```
//arbiter.cpp
#undef SC_INCLUDE_FX
#include "packet.h"    // Include packet header file
#include "arbiter.h"   // Include arbiter header file
#include <iostream>  // Include iostream for printing
// Arbiter functionality
void arbiter::func() {
  printf("Arbiter function started\n"); // Print statement
  sc_uint<1> v_connected_input[5];  // set when input is connected to an output
  sc_uint<1> v_reserved_output[6]; // set when output is reserved by a input (one
                                   // output more for simple coding)
  sc_uint<3> v_req[5];             // request signal for each input
  sc_uint<5> v_free;               // status of output in term of being free
  sc_uint<4> v_id;                 // ID of the arbiter
  sc_uint<5> v_arbit;              // arbitration result
  sc_uint<15> v_select;            // selection signal
  // Initialize variables
  for (int i = 0; i < 5; i++) {
    v_connected_input[i] = 0; // Initially no input is connected
    v_reserved_output[i] = 0;  // Initially no output is reserved
```

```
  v_req[i] = 0;              // Initially no request
}
v_free = 31;   // '11111' - Initially all outputs are free
v_arbit = 0;   // Initially no output is arbitrated
v_select = 0;  // Initially no selection

// functionality
while (true) { // Infinite loop
  wait();      // Wait for a clock cycle

  grant0.write(0); // reset grant for input 0
  grant1.write(0); // reset grant for input 1
  grant2.write(0); // reset grant for input 2
  grant3.write(0); // reset grant for input 3
  grant4.write(0); // reset grant for input 4
  // Update free output status
  if (!free_out0.read()) {
    v_free = v_free | 1;
  } // set the bit 0 showing the output 0 is free
  if (!free_out1.read()) {
    v_free = v_free | 2;
  }
  if (!free_out2.read()) {
    v_free = v_free | 4;
  }
  if (!free_out3.read()) {
    v_free = v_free | 8;
  }
  if (!free_out4.read()) {
    v_free = v_free | 16;
  }

  v_id = arbiter_id.read(); // Read arbiter ID

  // Process request from input 0
  if (!req0.read()[4]) // if FIFO buffer is not empty
  {
    // --- Torus XY Dimension-Order Routing Logic ---
    sc_uint<2> current_x = v_id.range(1,0);
    sc_uint<2> current_y = v_id.range(3,2);

    sc_uint<2> dest_x = req0.read().range(1,0);
    sc_uint<2> dest_y = req0.read().range(3,2);

    if (current_x != dest_x) { // Resolve X dimension first
        // Calculate shortest path distance (forward and backward) in X
        // For N=4 dimension size
        int dist_fwd_x = (int(dest_x) - int(current_x) + 4) % 4;
        int dist_bwd_x = (int(current_x) - int(dest_x) + 4) % 4;

        // Choose shortest path direction (prefer East/South in case of tie, dist=2)
        if (dist_fwd_x <= dist_bwd_x) {
            v_req[0] = 3; // Go East
        } else {
            v_req[0] = 5; // Go West
        }
    } else if (current_y != dest_y) { // X matches, resolve Y dimension
```

```
      // Calculate shortest path distance (forward and backward) in Y
      int dist_fwd_y = (int(dest_y) - int(current_y) + 4) % 4;
      int dist_bwd_y = (int(current_y) - int(dest_y) + 4) % 4;

      // Choose shortest path direction (prefer East/South in case of tie, dist=2)
      if (dist_fwd_y <= dist_bwd_y) {
          // Note: Assuming South corresponds to increasing Y
         v_req[0] = 4; // Go South
      } else {
         v_req[0] = 2; // Go North
      }
   } else { // X and Y match
      v_req[0] = 1; // Destination is Local
   }
  // Determine available output based on request
  switch (v_req[0]) {
  case 1:
    v_arbit = v_free & 1;
    break;
  case 2:
    v_arbit = v_free & 2;
    break;
  case 3:
    v_arbit = v_free & 4;
    break;
  case 4:
    v_arbit = v_free & 8;
    break;
  case 5:
    v_arbit = v_free & 16;
    break;
  default:
    break;
  }
  if (!v_connected_input[0]) // if input is not connected
  {
    if (v_reserved_output[v_req[0]])
      v_arbit = 0; // if the requested output was reserved, go to next input
  }
  if (v_arbit != 0) {
    grant0.write(1); // set grant
    v_select.range(2, 0) = v_req[0];
    v_free = v_free & (~v_arbit); // inactive the related output
    v_connected_input[0] = 1;      // input 0 is connected
    v_reserved_output[v_req[0]] = 1; // output is reserved
    if (req0.read()[5]) {
      v_connected_input[0] = 0;
      v_reserved_output[v_req[0]] =
          0;
    } // if it is tail flit, reset connection and reservation
  }
}
// Process request from input 1
if (!req1.read()[4]) // if buffer is not empty
{
  printf("Request received on input 1\n"); // Print statement
  // if(!v_connected_input[1]) // if input is not connected i.e. it is
```

```
// header
 // --- Torus XY Dimension-Order Routing Logic ---
 sc_uint<2> current_x = v_id.range(1,0);
 sc_uint<2> current_y = v_id.range(3,2);

 sc_uint<2> dest_x = req1.read().range(1,0);
 sc_uint<2> dest_y = req1.read().range(3,2);

 if (current_x != dest_x) { // Resolve X dimension first
    // Calculate shortest path distance (forward and backward) in X
    // For N=4 dimension size
    int dist_fwd_x = (int(dest_x) - int(current_x) + 4) % 4;
    int dist_bwd_x = (int(current_x) - int(dest_x) + 4) % 4;

    // Choose shortest path direction (prefer East/South in case of tie, dist=2)
    if (dist_fwd_x <= dist_bwd_x) {
       v_req[1] = 3; // Go East
    } else {
       v_req[1] = 5; // Go West
    }
 } else if (current_y != dest_y) { // X matches, resolve Y dimension
    // Calculate shortest path distance (forward and backward) in Y
    int dist_fwd_y = (int(dest_y) - int(current_y) + 4) % 4;
    int dist_bwd_y = (int(current_y) - int(dest_y) + 4) % 4;

    // Choose shortest path direction (prefer East/South in case of tie, dist=2)
    if (dist_fwd_y <= dist_bwd_y) {
        // Note: Assuming South corresponds to increasing Y
       v_req[1] = 4; // Go South
    } else {
       v_req[1] = 2; // Go North
    }
 } else { // X and Y match
    v_req[1] = 1; // Destination is Local
 }
// Determine available output based on request
switch (v_req[1]) {
case 1:
  v_arbit = v_free & 1;
  break;
case 2:
  v_arbit = v_free & 2;
  break;
case 3:
  v_arbit = v_free & 4;
  break;
case 4:
  v_arbit = v_free & 8;
  break;
case 5:
  v_arbit = v_free & 16;
  break;
default:
  break;
}
if (!v_connected_input[1]) // if input is not connected
{
```

```
    if (v_reserved_output[v_req[1]])
      v_arbit = 0; // if the requested output was reserved, go to next input
  }
  if (v_arbit != 0) { // if there is any free output
    grant1.write(1);  // set grant
    v_select.range(5, 3) = v_req[1];
    v_free = v_free & (~v_arbit); // inactive the related outputs
    v_connected_input[1] = 1;      // input 1 is connected
    v_reserved_output[v_req[1]] = 1; // output is reserved
    if (req1.read()[5]) {
      v_connected_input[1] = 0;
      v_reserved_output[v_req[1]] =
        0;
    } // if it is tail flit, reset connection and reservation
  }
}
// Process request from input 2
if (!req2.read()[4]) // if buffer is not empty
{
  // --- Torus XY Dimension-Order Routing Logic ---
  sc_uint<2> current_x = v_id.range(1,0);
  sc_uint<2> current_y = v_id.range(3,2);

  sc_uint<2> dest_x = req2.read().range(1,0);
  sc_uint<2> dest_y = req2.read().range(3,2);
  if (current_x != dest_x) { // Resolve X dimension first
     // Calculate shortest path distance (forward and backward) in X
     // For N=4 dimension size
     int dist_fwd_x = (int(dest_x) - int(current_x) + 4) % 4;
     int dist_bwd_x = (int(current_x) - int(dest_x) + 4) % 4;
     // Choose shortest path direction (prefer East/South in case of tie, dist=2)
     if (dist_fwd_x <= dist_bwd_x) {
        v_req[2] = 3; // Go East
     } else {
        v_req[2] = 5; // Go West
     }
  } else if (current_y != dest_y) { // X matches, resolve Y dimension
     // Calculate shortest path distance (forward and backward) in Y
     int dist_fwd_y = (int(dest_y) - int(current_y) + 4) % 4;
     int dist_bwd_y = (int(current_y) - int(dest_y) + 4) % 4;
     // Choose shortest path direction (prefer East/South in case of tie, dist=2)
     if (dist_fwd_y <= dist_bwd_y) {
        // Note: Assuming South corresponds to increasing Y
        v_req[2] = 4; // Go South
     } else {
        v_req[2] = 2; // Go North
     }
  } else { // X and Y match
     v_req[2] = 1; // Destination is Local
  }
  // Determine available output based on request
  switch (v_req[2]) {
  case 1:
    v_arbit = v_free & 1;
    break;
  case 2:
    v_arbit = v_free & 2;
```

```
    break;
  case 3:
    v_arbit = v_free & 4;
    break;
  case 4:
    v_arbit = v_free & 8;
    break;
  case 5:
    v_arbit = v_free & 16;
    break;
  default:
    break;
  }
  if (!v_connected_input[2]) // if input is not connected
  {
    if (v_reserved_output[v_req[2]])
      v_arbit = 0; // if the requested output was reserved, go to next input
  }
  if (v_arbit != 0) {
    grant2.write(1);  // set grant
    v_select.range(8, 6) = v_req[2];
    v_free = v_free & (~v_arbit); // inactive the related outputs
    v_connected_input[2] = 1;      // input 1 is connected
    v_reserved_output[v_req[2]] = 1; // output is reserved
    if (req2.read()[5]) {
      v_connected_input[2] = 0;
      v_reserved_output[v_req[2]] =
        0;
    } // if it is tail flit, reset connection and reservation
  }
}
// Process request from input 3
if (!req3.read()[4]) // if buffer is not empty
{
  printf("Request received on input 3\n"); // Print statement
  // if(!v_connected_input[3]) // if input is not connected i.e. it is
  // header
   // --- Torus XY Dimension-Order Routing Logic ---
   sc_uint<2> current_x = v_id.range(1,0);
   sc_uint<2> current_y = v_id.range(3,2);

   sc_uint<2> dest_x = req3.read().range(1,0);
   sc_uint<2> dest_y = req3.read().range(3,2);

   if (current_x != dest_x) { // Resolve X dimension first
      // Calculate shortest path distance (forward and backward) in X
      // For N=4 dimension size
      int dist_fwd_x = (int(dest_x) - int(current_x) + 4) % 4;
      int dist_bwd_x = (int(current_x) - int(dest_x) + 4) % 4;

      // Choose shortest path direction (prefer East/South in case of tie, dist=2)
      if (dist_fwd_x <= dist_bwd_x) {
         v_req[3] = 3; // Go East
      } else {
         v_req[3] = 5; // Go West
      }
   } else if (current_y != dest_y) { // X matches, resolve Y dimension
```

```
      // Calculate shortest path distance (forward and backward) in Y
      int dist_fwd_y = (int(dest_y) - int(current_y) + 4) % 4;
      int dist_bwd_y = (int(current_y) - int(dest_y) + 4) % 4;

      // Choose shortest path direction (prefer East/South in case of tie, dist=2)
      if (dist_fwd_y <= dist_bwd_y) {
          // Note: Assuming South corresponds to increasing Y
         v_req[3] = 4; // Go South
      } else {
         v_req[3] = 2; // Go North
      }
   } else { // X and Y match
      v_req[3] = 1; // Destination is Local
   }
  // Determine available output based on request
  switch (v_req[3]) {
  case 1:
    v_arbit = v_free & 1;
    break;
  case 2:
    v_arbit = v_free & 2;
    break;
  case 3:
    v_arbit = v_free & 4;
    break;
  case 4:
    v_arbit = v_free & 8;
    break;
  case 5:
    v_arbit = v_free & 16;
    break;
  default:
    break;
  }
  if (!v_connected_input[3]) // if input is not connected
  {
    if (v_reserved_output[v_req[3]])
      v_arbit = 0; // if the requested output was reserved, go to next input
  }
  if (v_arbit != 0) {
    grant3.write(1);  // set grant
    v_select.range(11, 9) = v_req[3];
    v_free = v_free & (~v_arbit); // inactive the related outputs
    v_connected_input[3] = 1;      // input 3 is connected
    v_reserved_output[v_req[3]] = 1; // output is reserved
    if (req3.read()[5]) {
      v_connected_input[3] = 0;
      v_reserved_output[v_req[3]] =
        0;
    } // if it is tail flit, reset connection and reservation
  }
}
// Process request from input 4
if (!req4.read()[4]) // if buffer is not empty
{
  printf("Request received on input 4\n"); // Print statement
  // if(!v_connected_input[4]) // if input is not connected i.e. it is
```

```
// header
 // --- Torus XY Dimension-Order Routing Logic ---
 sc_uint<2> current_x = v_id.range(1,0);
 sc_uint<2> current_y = v_id.range(3,2);

 sc_uint<2> dest_x = req4.read().range(1,0);
 sc_uint<2> dest_y = req4.read().range(3,2);

 if (current_x != dest_x) { // Resolve X dimension first
    // Calculate shortest path distance (forward and backward) in X
    // For N=4 dimension size
    int dist_fwd_x = (int(dest_x) - int(current_x) + 4) % 4;
    int dist_bwd_x = (int(current_x) - int(dest_x) + 4) % 4;

    // Choose shortest path direction (prefer East/South in case of tie, dist=2)
    if (dist_fwd_x <= dist_bwd_x) {
       v_req[4] = 3; // Go East
    } else {
       v_req[4] = 5; // Go West
    }
 } else if (current_y != dest_y) { // X matches, resolve Y dimension
    // Calculate shortest path distance (forward and backward) in Y
    int dist_fwd_y = (int(dest_y) - int(current_y) + 4) % 4;
    int dist_bwd_y = (int(current_y) - int(dest_y) + 4) % 4;

    // Choose shortest path direction (prefer East/South in case of tie, dist=2)
    if (dist_fwd_y <= dist_bwd_y) {
        // Note: Assuming South corresponds to increasing Y
       v_req[4] = 4; // Go South
    } else {
       v_req[4] = 2; // Go North
    }
 } else { // X and Y match
    v_req[4] = 1; // Destination is Local
 }
// Determine available output based on request
switch (v_req[4]) {
case 1:
  v_arbit = v_free & 1;
  break;
case 2:
  v_arbit = v_free & 2;
  break;
case 3:
  v_arbit = v_free & 4;
  break;
case 4:
  v_arbit = v_free & 8;
  break;
case 5:
  v_arbit = v_free & 16;
  break;
default:
  break;
}
if (!v_connected_input[4]) // if input is not connected
{
```

```
      if (v_reserved_output[v_req[4]])
        v_arbit = 0; // if the requested output was reserved, go to next input
      }
      if (v_arbit != 0) {
        grant4.write(1);  // set grant
        v_select.range(14, 12) = v_req[4];
        v_free = v_free & (~v_arbit); // inactive the related outputs
                              printf("Selection signal written: %d\n", (int)v_select); // Print statement
        v_connected_input[4] = 1;      // input 4 is connected
        v_reserved_output[v_req[4]] = 1; // output is reserved
        if (req4.read()[5]) {
          v_connected_input[4] = 0;
          v_reserved_output[v_req[4]] =
            0;
        } // if it is tail flit, reset connection and reservation
      }
    }
    aselect.write(v_select); // Write the selection signal

  }
  printf("Arbiter function ended\n"); // Print statement
}
```